iscte INSTITUTO UNIVERSITÁRIO DE LISBOA


## Advanced Topics in Deep Learning

**FINAL PROJECT**

# ADVANCED TIME SERIES

## FORECASTING, OPTIMIZATION AND EXPLAINABILITY

**António Cruz** (140129), **Cátia Brás** (120093), **Ricardo Kayseller** (95813)

# Table of Contents

1. Introduction and Project Goals ..... 5
2. Environment, Reproducibility and Experimental Setup ..... 7
2.1 Libraries Import ..... 7
2.2 Configuration and Utils ..... 7
3. Dataset and Problem Definition ..... 9
3.1 Load Data ..... 9
3.2 Initial Dataset Inspection ..... 9
3.3 Datetime Parsing and Temporal Ordering ..... 10
3.4 Hourly Resampling ..... 10
3.5 Post-Resampling Quality Check ..... 11
3.6 Handling Missing Values After Resampling ..... 11
3.7 Forecasting Task and Variable Selection ..... 11
4. Data Initial Preparation ..... 13
4.1 Target Definition and Base Modeling DataFrame ..... 13
4.2 Base Forecasting Data Overview ..... 13
5. Feature Engineering ..... 14
5.1 Cyclical Time Features ..... 14
5.2 Wind-Derived Features ..... 14
5.3 Final Feature Set for Modeling ..... 15
6. Split, Scaling and Windowing ..... 16
6.1 Temporal Train/Validation/Test Split ..... 16
6.2 Feature Scaling ..... 16
6.3 Target Index for Forecasting ..... 16
6.4 Supervised Windowing ..... 16
6.5 Windowed Data Sanity Check ..... 17
7. Baseline Models ..... 18
7.1 Persistence Baseline ..... 18
7.2 GRU Baseline ..... 19
7.3 Baseline Comparison ..... 21
7.4 Reverse Scaling ..... 21
7.5 Baseline Evaluation in Original Temperature Scale ..... 21
7.6 Final Baseline Comparison ..... 22
7.7 Forecast Visualization ..... 22
8. Synthetic Data Generation with TimeGAN ..... 24
8.1 Motivation and Experimental Role ..... 24

8.2 TimeGAN Training Data Preparation ..... 24
8.3 TimeGAN Model and Training Procedure ..... 24
8.4 Synthetic Sequence Quality Assessment ..... 28
9. Evolutionary Optimization of the Forecasting Pipeline ..... 31
9.1 Motivation and Search Strategy ..... 31
9.2 Search Space Definition ..... 31
9.3 Evolutionary Representation and Constraints ..... 32
9.4 Fitness Function ..... 33
9.5 Evolutionary Search Execution ..... 33
9.6 Best Configuration and Retraining ..... 34
9.7 Top Candidate Configurations ..... 35
10. Retraining and Final Model Selection ..... 37
10.1 Selection of Final Candidates ..... 37
10.2 Retraining Procedure ..... 38
10.3 Final Test Set Evaluation ..... 41
10.4 Final Model Comparison ..... 42
10.5 Final Model Selection ..... 43
10.6 Robustness Across Random Seeds ..... 43
11. Explainable AI ..... 44
11.1 Global Explainability ..... 44
11.2 Local Explainability ..... 45
11.3 XAI Summary ..... 48
11.4 XAI-Guided Feature Pruning ..... 48
11.5 Effect of XAI-Guided Feature Pruning ..... 51
12. Efficiency, Latency and Resource Analysis ..... 52
12.1 Motivation ..... 52
12.2 Models Selected for Efficiency Analysis ..... 52
12.3 Training Time ..... 52
12.4 Memory Usage (RAM / GPU) ..... 52
12.5 Trainable Parameters ..... 53
12.6 Inference Latency ..... 53
12.7 Efficiency Comparison ..... 54
12.8 Final Efficiency Discussion ..... 56
13. Comparative Discussion ..... 58
Summary of Results ..... 58
Baseline vs. Evolutionary Performance ..... 58
Impact of Evolutionary Optimization ..... 58

Robustness Validation ..... 59
Explainability Insights ..... 59
Accuracy–Efficiency Trade-offs ..... 59
Strengths and Limitations ..... 60
14. Future Work ..... 61
15. Conclusion ..... 62
Key Findings ..... 62
XAI-Driven Model Refinement ..... 62
Final Assessment ..... 63

# 1. Introduction and Project Goals

This project builds upon the Time Series Modelling mini-project to develop a systematic, optimized, and well-analysed forecasting system for air temperature prediction using the Jena Climate dataset. While the mini-project focused on exploratory modelling and understanding different design choices, this Final Project elevates performance to a primary objective, achieved through systematic optimization and supported by rigorous analysis.

The core forecasting task is a **multivariate-input, univariate-output, multi-step prediction** problem: given a window of historical meteorological measurements (temperature, pressure, humidity, wind speed, maximum wind speed, and wind direction), the system predicts the air temperature for the next 24 hours.

Our pipeline integrates the following advanced techniques:

- **Baseline Deep Learning Model (GRU):** A configurable multi-layer GRU architecture serves as the reference model against which all improvements are measured, incorporating regularization, gradient clipping, and modern training strategies (AdamW optimizer, Huber loss).
- **Evolutionary Optimization:** A genetic algorithm (GA) automatically searches over an end-to-end pipeline configuration space - jointly optimizing model hyperparameters, architectural choices, training settings, preprocessing strategy, regularization, and windowing parameters. With a population of 20 individuals over 15 generations (300 total evaluations), the EA discovers configurations that outperform the hand-tuned baseline while using fewer parameters.
- **Synthetic Data Generation (TimeGAN):** A Time-series Generative Adversarial Network generates realistic synthetic weather sequences from the training set, demonstrating the feasibility of data augmentation without information leakage.
- **Explainable AI (XAI):** Both global (permutation importance) and local (gradient saliency) explanation methods are applied. XAI insights are further used to guide **feature pruning**, removing redundant features to produce a simpler model with improved performance and robustness.
- **Efficiency and Resource Analysis:** Training time, inference latency, parameter counts, and memory usage (RAM and GPU) are profiled across model configurations, enabling an informed analysis of accuracy–efficiency trade-offs.

The remainder of this report is organized as follows:

- Section 2 describes our experimental setup and reproducibility measures.
- Section 3 presents the dataset and problem formulation.
- Sections 4–6 cover data preparation, feature engineering, and the windowing pipeline.

- Section 7 establishes baseline models.
- Section 8 details the Time GAN synthetic data generation.
- Section 9 presents the evolutionary optimization process.
- Section 10 covers final model retraining, selection, and robustness validation.
- Sections 11–12 present the explainability and efficiency analyses.
- Sections 13–14 provide the comparative discussion and conclusion.

## 2. Environment, Reproducibility and Experimental Setup

This section establishes the computational environment, library dependencies, random seed configuration, and project structure. Reproducibility is ensured through fixed seeds, explicit dependency management, and modular code organization.

### 2.1 Libraries Import

```
import os
import gc
import sys
import json
import random
import time
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

from sklearn.preprocessing import StandardScaler, RobustScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

import optuna
import psutil
import subprocess
```

We verify that TensorFlow detects the available GPU and configure memory growth to prevent out-of-memory errors during the evolutionary search, which trains many models sequentially.

### 2.2 Configuration and Utils

We also set a global random seed (SEED = 42) applied consistently to Python, NumPy, and TensorFlow via set_global_seed() to ensure reproducibility. GPU memory growth is enabled via enable_gpu_memory_growth(), and device information is printed to confirm the hardware configuration.

```
SEED = 42

os.environ["PYTHONHASHSEED"] = str(SEED)
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
```

Set up the project root path and import custom utility functions.

```
PROJECT_ROOT = Path.cwd().parent
if str(PROJECT_ROOT) not in sys.path:
    sys.path.append(str(PROJECT_ROOT))

print(PROJECT_ROOT)
/home/ricarl/taap_p4
```

We import our custom utility functions and verify the environment setup (seed, GPU memory growth, device info).

```
from src.utils.env import set_global_seed, enable_gpu_memory_growth, get_device_i
nfo

SEED = 42
set_global_seed(SEED)
enable_gpu_memory_growth()

print(get_device_info())
{'tensorflow_version': '2.21.0', 'num_gpus': 1, 'gpus': ['/physical_device:GPU:0
'], 'num_cpus': 1, 'cpus': ['/physical_device:CPU:0']}
```

# 3. Dataset and Problem Definition

The Jena Climate dataset serves as the foundation for all experiments. This section covers data loading, quality assurance, temporal resampling to hourly resolution, and the formal definition of the forecasting task and variable selection.

## 3.1 Load Data

The Jena Climate dataset is loaded from a local CSV file. The dataset contains meteorological measurements collected at a weather station in Jena, Germany, between January 2009 and December 2016, with an original sampling frequency of 10 minutes across 15 variables.

## 3.2 Initial Dataset Inspection

Upon loading, we inspect the dataset shape, column names, data types, and general structure. This provides a first look at the raw data before any transformations are applied.

**Table 1** - First rows of the raw Jena Climate dataset (10-minute sampling, 15 variables).

```
 #   Column           Non-Null Count   Dtype
---  ------           --------------   -----
 0   Date Time        420551 non-null  str
 1   p (mbar)         420551 non-null  float64
 2   T (degC)         420551 non-null  float64
 3   Tpot (K)         420551 non-null  float64
 4   Tdew (degC)      420551 non-null  float64
 5   rh (%)           420551 non-null  float64
 6   VPmax (mbar)     420551 non-null  float64
 7   VPact (mbar)     420551 non-null  float64
 8   VPdef (mbar)     420551 non-null  float64
 9   sh (g/kg)        420551 non-null  float64
 10  H2OC (mmol/mol)  420551 non-null  float64
 11  rho (g/m**3)     420551 non-null  float64
 12  wv (m/s)         420551 non-null  float64
 13  max. wv (m/s)    420551 non-null  float64
 14  wd (deg)         420551 non-null  float64
dtypes: float64(14), str(1)
memory usage: 48.1 MB
```

After loading the raw dataset, we perform quality checks to ensure data integrity before any transformations. Specifically, we verify the presence of duplicated rows and inspect all columns for missing values. Any duplicated observations are removed to avoid biasing the resampling step that follows - duplicate timestamps can arise from logging artifacts and must be eliminated to ensure a consistent temporal index.

## 3.3 Datetime Parsing and Temporal Ordering

The Date Time column is parsed into proper datetime objects and the dataframe is sorted chronologically. We verify the date range and inspect the time delta distribution to detect any irregularities in the 10-minute sampling frequency before resampling.

**Table 2** - Dataset date range and first rows after datetime parsing and temporal ordering.

| | Date Time |
|---|---|
| 0 | 2009-01-01 00:10:00 |
| 1 | 2009-01-01 00:20:00 |
| 2 | 2009-01-01 00:30:00 |
| 3 | 2009-01-01 00:40:00 |
| 4 | 2009-01-01 00:50:00 |

Check the time delta distribution to identify sampling irregularities and inspect the distribution of time deltas between consecutive observations to detect any sampling irregularities.

```
df["Date Time"].diff().value_counts().head(10)
Date Time
0 days 00:10:00    420218
0 days 00:20:00         2
0 days 00:30:00         1
0 days 16:00:00         1
3 days 02:20:00         1
Name: count, dtype: int64
```

## 3.4 Hourly Resampling

After removing duplicated rows, the cleaned dataset contained 420,224 observations at approximately 10-minute resolution. To align the forecasting problem with hourly dynamics and reduce computational cost, the series was resampled to **1-hour intervals** using resample("1h").mean(), which aggregates all observations within each hourly bin through arithmetic averaging.

This operation reduced the dataset from **420,224** rows to **70,129** rows, approximately a sixfold reduction, while preserving the dominant meteorological patterns relevant for day-ahead forecasting. Mean aggregation was preferred over alternatives such as first/last value selection or median aggregation because it preserves the central tendency of each hourly period while smoothing sub-hourly noise that is less relevant for the 24-hour prediction horizon.

## 3.5 Post-Resampling Quality Check

After hourly resampling, the dataset was re-validated to ensure that the transformation did not introduce structural artifacts. In particular, we verified that no duplicated hourly timestamps were created, that the time difference between consecutive observations was predominantly **1 hour**, and that any missing values produced by incomplete hourly bins were explicitly identified.

The quality check showed that the resampled dataset contained **88 missing values per meteorological variable**, while the Date Time column remained complete. No duplicated timestamps were found after resampling, and the time-step distribution confirmed that the series was overwhelmingly regular at **1-hour intervals**.

This validation step is essential because the supervised sliding-window procedure used later assumes a temporally ordered and nearly regular time series.

## 3.6 Handling Missing Values After Resampling

All rows containing NaN values after hourly resampling were removed. Because the original dataset has near-complete 10-minute coverage across the 2009–2016 period, the number of affected rows was very small relative to the full hourly dataset. After dropping these rows, the dataset size became **70,041 observations**, and all remaining missing values were eliminated.

Dropping these observations was preferred over imputation because the affected hourly bins were sparse and associated with incomplete aggregation windows or isolated temporal irregularities. In such cases, interpolation could introduce artificial patterns into the series and potentially bias the forecasting models.

Although the cleaned series still contains a very small number of larger temporal gaps, the dataset remains overwhelmingly regular at hourly frequency and is suitable for the supervised windowing procedure used in the next stages.

## 3.7 Forecasting Task and Variable Selection

Following the project specification, we retain 6 meteorological variables plus the datetime reference: T (degC) (air temperature, the target), p (mbar) (atmospheric pressure), rh (%) (relative humidity), wv (m/s) (wind speed), max. wv (m/s) (maximum wind speed), and wd (deg) (wind direction). All other variables from the original 15-column dataset are excluded.

This defines a **multivariate-input, univariate-output, multi-step** forecasting problem: the model receives a multi-day window of multi-sensor weather data and must predict the air temperature trajectory for the next 24 hours.

The multivariate input ensures the model can leverage cross-variable dependencies - for instance, the relationship between falling pressure and subsequent temperature changes driven by weather fronts.

**Table 3** - Selected variables for the forecasting task (6 meteorological features + datetime).

| | Date Time | T (degC) | p (mbar) | rh (%) | wv (m/s) | max. wv (m/s) | wd (deg) |
|---|---|---|---|---|---|---|---|
| 0 | 2009-01-01 00:00:00 | -8.304000 | 996.528000 | 93.780000 | 0.520000 | 1.002000 | 174.460000 |
| 1 | 2009-01-01 01:00:00 | -8.065000 | 996.525000 | 93.933333 | 0.316667 | 0.711667 | 172.416667 |
| 2 | 2009-01-01 02:00:00 | -8.763333 | 996.745000 | 93.533333 | 0.248333 | 0.606667 | 196.816667 |
| 3 | 2009-01-01 03:00:00 | -8.896667 | 996.986667 | 93.200000 | 0.176667 | 0.606667 | 157.083333 |
| 4 | 2009-01-01 04:00:00 | -9.348333 | 997.158333 | 92.383333 | 0.290000 | 0.670000 | 150.093333 |

# 4. Data Initial Preparation

Before applying feature engineering, the core modelling dataframe is established by defining the target variable and confirming the structure and integrity of the cleaned dataset. This section bridges the data cleaning stage presented in Section 3 with the feature engineering pipeline developed in Section 5, ensuring a clear and consistent transition from raw observations to model-ready inputs.

## 4.1 Target Definition and Base Modeling DataFrame

The target variable is defined as T (degC) - air temperature in degrees Celsius. This is the only quantity the model must forecast in the output window; all other variables serve as exogenous inputs.

The Date Time column serves as the temporal reference for feature engineering (extracting hour-of-day and day-of-year), but is not included as a direct model input since neural networks cannot interpret raw datetime objects. Instead, its temporal information is encoded as cyclic features in Section 5.

The remaining 5 meteorological variables - atmospheric pressure, relative humidity, wind speed, maximum wind speed, and wind direction - form the covariate set that provides the atmospheric context for the temperature forecast.

## 4.2 Base Forecasting Data Overview

We inspect the prepared dataframe to confirm shape, date range, and column integrity before proceeding to feature engineering. The dataset at this stage contains approximately 70,000 hourly observations spanning from January 2009 to January 2017, with 7 columns (datetime + 6 meteorological variables). This verification step ensures that no data was inadvertently lost during the resampling and cleaning pipeline of Section 3, and establishes the baseline from which all derived features will be computed.

# 5. Feature Engineering

Two families of derived features are introduced to enrich the input representation: **cyclical temporal encodings** and **wind-derived features**. The goal is to provide the neural network with representations that respect the physical structure of the data, cyclic quantities are encoded cyclically, and vector quantities are decomposed into Cartesian components.

All feature engineering is implemented in src.features.engineering for reproducibility.

## 5.1 Cyclical Time Features

Hour-of-day and day-of-year carry strong periodic signals (diurnal and seasonal cycles). Encoding them as raw integers would introduce artificial discontinuities (e.g., hour 23 far from hour 0). We apply a sine–cosine encoding that maps each cyclic variable to a point on the unit circle, ensuring smooth transitions at boundaries:

- hour_sin, hour_cos
- diurnal cycle (period = 24h)
- doy_sin, doy_cos
- seasonal cycle (period = 365.25 days)

**Table 4** - Cyclical temporal features derived from hour-of-day and day-of-year.

| | Date Time | hour | dayofyear | hour_sin | hour_cos | doy_sin | doy_cos |
|---|---|---|---|---|---|---|---|
| 0 | 2009-01-01 00:00:00 | 0 | 1 | 0.000000 | 1.000000 | 0.017202 | 0.999852 |
| 1 | 2009-01-01 01:00:00 | 1 | 1 | 0.258819 | 0.965926 | 0.017202 | 0.999852 |
| 2 | 2009-01-01 02:00:00 | 2 | 1 | 0.500000 | 0.866025 | 0.017202 | 0.999852 |
| 3 | 2009-01-01 03:00:00 | 3 | 1 | 0.707107 | 0.707107 | 0.017202 | 0.999852 |
| 4 | 2009-01-01 04:00:00 | 4 | 1 | 0.866025 | 0.500000 | 0.017202 | 0.999852 |

## 5.2 Wind-Derived Features

Wind direction in degrees is inherently circular and difficult for neural networks to interpret directly.

We derive six features:

- wd_sin, wd_cos - cyclic encoding of wind direction
- wx, wy - Cartesian decomposition of wind vector (speed × cos/sin of direction), replacing the polar representation with a form neural networks process more naturally
- wind_gap - difference between max and sustained wind speed (gust intensity)
- gust_ratio - ratio of max to sustained wind speed (relative gust strength, with $\varepsilon = 10^{-6}$ for numerical stability)

**Table 5** - Wind-derived features: cyclic encoding, Cartesian components, gust metrics.

| Date Time | wv (m/s) | max. wv (m/s) | wd (deg) | wd_sin | wd_cos | wx | wy | wind_gap | gust_ratio |
|---|---|---|---|---|---|---|---|---|---|
| 2009-01-01 00:00:00 | 0.520000 | 1.002000 | 174.460000 | 0.096541 | -0.995329 | -0.517571 | 0.050201 | 0.482000 | 1.926919 |
| 2009-01-01 01:00:00 | 0.316667 | 0.711667 | 172.416667 | 0.131968 | -0.991254 | -0.313897 | 0.041790 | 0.395000 | 2.247361 |
| 2009-01-01 02:00:00 | 0.248333 | 0.606667 | 196.816667 | -0.289310 | -0.957235 | -0.237713 | -0.071845 | 0.358333 | 2.442943 |
| 2009-01-01 03:00:00 | 0.176667 | 0.606667 | 157.083333 | 0.389392 | -0.921072 | -0.162723 | 0.068793 | 0.430000 | 3.433943 |
| 2009-01-01 04:00:00 | 0.290000 | 0.670000 | 150.093333 | 0.498589 | -0.866839 | -0.251383 | 0.144591 | 0.380000 | 2.310337 |

## 5.3 Final Feature Set for Modeling

The complete input feature vector contains **16 dimensions**: 6 original meteorological variables + 4 cyclical temporal encodings + 6 wind-derived features. This feature set is used consistently across all experiments, baseline, evolutionary optimization, and synthetic data generation. The XAI analysis in Section 11 later evaluates which of these 16 features contribute most, leading to a pruned 11-feature variant.

```
final_feature_cols = get_final_feature_columns()

print("Number of modeling features:", len(final_feature_cols))
print(final_feature_cols)
Number of modeling features: 16
['T (degC)', 'p (mbar)', 'rh (%)', 'wv (m/s)', 'max. wv (m/s)', 'wd (deg)', 'hour
_sin', 'hour_cos', 'doy_sin', 'doy_cos', 'wd_sin', 'wd_cos', 'wx', 'wy', 'wind_ga
p', 'gust_ratio']
```

# 6. Split, Scaling and Windowing

This section completes the data preparation pipeline by defining the temporal train/validation/test split, applying feature scaling without data leakage, and transforming the time series into supervised learning windows. These steps produce the final input–output tensors used by all forecasting models throughout the project.

## 6.1 Temporal Train/Validation/Test Split

The dataset is split chronologically (70% / 15% / 15%) with **no shuffling**, the split respects temporal order to prevent data leakage. The training set covers the earliest years, followed by validation, with the most recent data reserved for testing. This mirrors a realistic deployment scenario where the model is trained on historical data and evaluated on future observations.

## 6.2 Feature Scaling

All input features are normalized using **StandardScaler** (zero mean, unit variance) as the baseline strategy. The scaler is fit exclusively on the training set and applied via transform to validation and test sets, preventing information leakage. The evolutionary optimization also searches over the scaler type (standard, robust, minmax), allowing the GA to discover whether a different normalization strategy benefits specific model configurations.

We apply the selected scaler to all splits (fit on train, transform on val/test), and apply the scaler to the training, validation, and test feature matrices. The scaler is fit only on the training data.

## 6.3 Target Index for Forecasting

We identify the position of T (degC) within the 16-feature vector. This index is used during windowing to extract only the temperature column for the output sequence **y**, while keeping all 16 features in the input tensor **X**. This separation makes the problem multivariate-input but univariate-output.

## 6.4 Supervised Windowing

The scaled multivariate time series is converted into supervised learning samples through a sliding-window procedure. For each sample, the input tensor **X** contains a sequence of consecutive hourly observations across all modeling features, while the output vector **y** contains only the future air temperature values to be predicted.

Under the default configuration, the input has shape **(LOOKBACK, 16)** and the output has shape **(HORIZON,)**, corresponding to a multivariate-input, univariate-output, multi-step forecasting setup. In the base configuration, LOOKBACK = 120 hours (5 days) and HORIZON = 24 hours (1 day ahead). In the evolutionary optimization stage presented in Section 9, alternative lookback lengths are also explored as part of the search space, so the windowing strategy is not treated as fixed throughout the project.

We create supervised windows for all splits using the defined lookback and horizon, and create supervised windows for train, validation, and test splits using the defined lookback (120h) and horizon (24h).

## 6.5 Windowed Data Sanity Check

A final sanity check is performed on the windowed data to confirm that the supervised transformation produced tensors with the expected structure and dimensionality. In particular, the input windows follow the shape **(N, LOOKBACK, 16)**, while the target arrays follow the shape **(N, HORIZON)**, corresponding to multivariate input sequences and 24-step univariate temperature targets.

The inspection confirms that each input sample contains **120 hourly observations across 16 features**, and that each target sample contains **24 future temperature values**. This validation step is important because it helps detect off-by-one errors or alignment mistakes in the windowing process that could silently compromise model training and evaluation.

```
print("Input shape:", X_train.shape[1:])
print("Forecast horizon:", y_train.shape[1])
print("Number of input features:", X_train.shape[2])

print("\nExample input window shape:", X_train[0].shape)
print("Example target shape:", y_train[0].shape)
print("First 5 target values (scaled):", y_train[0][:5])
Input shape: (120, 16)
Forecast horizon: 24
Number of input features: 16

Example input window shape: (120, 16)
Example target shape: (24,)
First 5 target values (scaled): [-2.55218147 -2.61156348 -2.64453206 -2.68540539
-2.73726824]
```

# 7. Baseline Models

Sections 7 to 9 present the core experimental pipeline of the project, covering baseline establishment, synthetic data generation, and evolutionary optimization.

Two baseline models are defined as reference points for all subsequent analyses. The **persistence baseline** provides a minimal no-skill benchmark that any learned forecasting model should outperform. The **GRU baseline** represents a stronger deep learning reference model, trained under a modern and carefully controlled setup, and serves as the main benchmark against which the optimized evolutionary solutions are evaluated.

## 7.1 Persistence Baseline

The persistence baseline corresponds to the simplest possible forecasting strategy: the last observed temperature value is repeated across the entire 24-hour forecast horizon. This naive approach provides a minimal reference level for performance, since any learned forecasting model should be able to outperform it.

The resulting prediction tensor has shape **(10364, 24)**, matching the multi-step forecasting target, and each prediction consists of a constant repetition of the final observed temperature from the corresponding input window.

Compute scaled metrics for the persistence forecast on the test set.

```
print("Persistence prediction shape:", y_pred_persistence.shape)
print("First prediction:", y_pred_persistence[0][:5])
Persistence prediction shape: (10364, 24)
First prediction: [-0.04174932 -0.04174932 -0.04174932 -0.04174932 -0.04174932]
```

Compute persistence baseline metrics on the test set.

```
print("Persistence MAE (scaled):", persistence_mae_scaled)
print("Persistence RMSE (scaled):", persistence_rmse_scaled)
Persistence MAE (scaled): 0.3636531434000367
Persistence RMSE (scaled): 0.4921206134926399
```

Before building the official configurable baseline, we construct a simple 1-layer GRU (64 units) as a quick sanity check.

This verifies that the full pipeline - from windowed data through model training to test evaluation - works correctly end-to-end. The simple model's results are not used for comparison; they serve only as a development checkpoint.

After training  the simple GRU with early stopping on validation loss, we generate and inspect test set predictions from the simple GRU and Evaluate the simple GRU on scaled metrics to compare against persistence.

```
print("Prediction shape:", y_pred_gru.shape)
print("First prediction:", y_pred_gru[0][:5])
Prediction shape: (10364, 24)
First prediction: [-0.13829982 -0.19688366 -0.15950276 -0.10881609 -0.00536779]
```

```
print("GRU MAE (scaled):", gru_mae_scaled)
print("GRU RMSE (scaled):", gru_rmse_scaled)
GRU MAE (scaled): 0.19162778158532956
GRU RMSE (scaled): 0.25370004555151926
```

## 7.2 GRU Baseline

The official GRU baseline was defined using the best-performing configuration identified in the previous Time Series Modelling mini-project. This ensures methodological continuity and provides a strong, previously validated reference model for the current project.

The model is implemented through build_gru_model() from src.models.gru and consists of a configurable 2-layer GRU architecture with **96** and **64** hidden units, respectively. The baseline also includes an intermediate dense layer with **256** units, **AdamW** optimization, **Huber loss** with ( = 1.0 ), **L2 regularization**, and **gradient clipping**. Training is performed with **early stopping** (patience=6) and **learning rate reduction on plateau** (patience=3, factor=0.5), providing a stable and competitive deep learning benchmark against which the evolutionary optimization will be evaluated.

Define the official baseline configuration and build the model.

```
from src.models.gru import build_gru_model
```

```
BASELINE_CFG = {
    "n_layers": 2,
    "units1": 96,
    "units2": 64,
    "units3": 96,
    "dropout": 0.0,
    "l2": 1e-6,
    "dense_units": 256,
    "dense_activation": "relu",
    "learning_rate": 2e-4,
    "clipnorm": 2.0,
    "optimizer_name": "adamw",
    "weight_decay": 1e-6,
    "loss_name": "huber1",
    "gaussian_noise_std": 0.0,
    "batch_size": 128,
```

```
}

gru_baseline = build_gru_model(
    L=LOOKBACK,
    n_features=X_train.shape[2],
    H=HORIZON,
    units1=BASELINE_CFG["units1"],
    units2=BASELINE_CFG["units2"],
    units3=BASELINE_CFG["units3"],
    n_layers=BASELINE_CFG["n_layers"],
    dropout=BASELINE_CFG["dropout"],
    l2=BASELINE_CFG["l2"],
    dense_units=BASELINE_CFG["dense_units"],
    dense_activation=BASELINE_CFG["dense_activation"],
    learning_rate=BASELINE_CFG["learning_rate"],
    clipnorm=BASELINE_CFG["clipnorm"],
    optimizer_name=BASELINE_CFG["optimizer_name"],
    weight_decay=BASELINE_CFG["weight_decay"],
    loss_name=BASELINE_CFG["loss_name"],
    gaussian_noise_std=BASELINE_CFG["gaussian_noise_std"],
)

gru_baseline.summary()
from src.models.train_eval import (
    train_model,
    evaluate_scaled_forecasts,
    inverse_scale_target,
    evaluate_original_scale_forecasts,
)
```

The official baseline is trained with the full 60-epoch budget. Training uses early stopping (patience=6, restoring best weights) and learning rate reduction on plateau (patience=3, factor=0.5), ensuring the model converges without overfitting.

We generate test predictions from the official baseline, inspect the output and compute scaled MAE and RMSE for the official GRU baseline on the test set:

```
print("Prediction shape:", y_pred_gru.shape)
print("First prediction:", y_pred_gru[0][:12])
Prediction shape: (10364, 24)
First prediction: [-0.12780994 -0.14077106 -0.10512683 -0.07985469 -0.02349082
0.0554703
  0.1347603   0.26358065  0.2870692   0.31509727  0.29663756  0.222746  ]
```

```
print("GRU Baseline MAE (scaled):", gru_mae_scaled)
print("GRU Baseline RMSE (scaled):", gru_rmse_scaled)
GRU Baseline MAE (scaled): 0.19262296176021332
GRU Baseline RMSE (scaled): 0.2555149343830984
```

## 7.3 Baseline Comparison

Table 6 presents a side-by-side comparison of the persistence baseline and the official GRU baseline using **scaled evaluation metrics**, namely MAE and RMSE computed in normalized space. These metrics are useful for comparing models trained under the same scaling regime and for monitoring optimization behaviour during training.

However, scaled errors do not have a direct physical interpretation. For this reason, the comparison is extended in the following subsections by converting both predictions and targets back to the original temperature scale, allowing the results to be interpreted in degrees Celsius.

**Table 6** - Persistence vs. GRU baseline comparison on scaled metrics (test set).

| | Model | MAE_scaled | RMSE_scaled |
|---|---|---|---|
| 0 | Persistence | 0.363653 | 0.492121 |
| 1 | GRU Baseline Official | 0.192623 | 0.255515 |

## 7.4 Reverse Scaling

Predictions and ground truth are inverse-transformed from normalized space back to degrees Celsius using the training set's scaler statistics (mean and standard deviation of the temperature column). This allows evaluation in physically meaningful units.

```
print("y_test_inv shape:", y_test_inv.shape)
print("y_pred_gru_inv shape:", y_pred_gru_inv.shape)
y_test_inv shape: (10364, 24)
y_pred_gru_inv shape: (10364, 24)
```

## 7.5 Baseline Evaluation in Original Temperature Scale

We compute MAE and RMSE in degrees Celsius on the inverse-scaled predictions. These original-scale metrics are the primary comparison basis throughout the report - an MAE of 1.65°C means the model's 24-hour forecasts are off by an average of 1.65 degrees. This is the metric used as the evolutionary fitness function and the final evaluation criterion.

```
print("GRU Baseline MAE (°C):", gru_mae)
print("GRU Baseline RMSE (°C):", gru_rmse)
GRU Baseline MAE (°C): 1.6651472981896625
GRU Baseline RMSE (°C): 2.208822866947564
```

## 7.6 Final Baseline Comparison

Table 7, combines both **scaled** and **original-scale** evaluation metrics for the persistence and GRU baseline models. The official GRU baseline substantially outperforms the naive persistence forecast, reducing the test error from **3.144 °C** to **1.665 °C** in MAE and from **4.254 °C** to **2.209 °C** in RMSE.

These results confirm that the recurrent architecture is able to capture meaningful temporal dependencies in the meteorological series and establish a strong benchmark against which the subsequent optimization stages will be evaluated.

**Table 7** - Final baseline comparison with both scaled and original-scale (°C) metrics.

| | Model | MAE_scaled | RMSE_scaled | MAE_degC | RMSE_degC |
|---|---|---|---|---|---|
| 0 | Persistence | 0.363653 | 0.492121 | 3.143634 | 4.254183 |
| 1 | GRU Baseline Official | 0.192623 | 0.255515 | 1.665147 | 2.208823 |

## 7.7 Forecast Visualization

Figure 1, presents a visual comparison of a single 24-hour forecast window on the test set, showing the true future temperature trajectory together with the persistence and GRU baseline predictions in degrees Celsius.

The persistence baseline produces a flat forecast, since it simply repeats the last observed temperature across the full horizon. As expected, this strategy fails to capture the strong upward and downward variations visible in the true trajectory. In contrast, the GRU baseline is able to follow the general temporal pattern more closely, correctly anticipating the broad rise in temperature during the first half of the horizon and the subsequent declining trend.

Although the GRU forecast is smoother than the true signal and underestimates the amplitude of some changes, it still represents a substantial improvement over persistence. This qualitative comparison is consistent with the quantitative results reported earlier and confirms that the recurrent model captures meaningful temporal structure in the meteorological series.

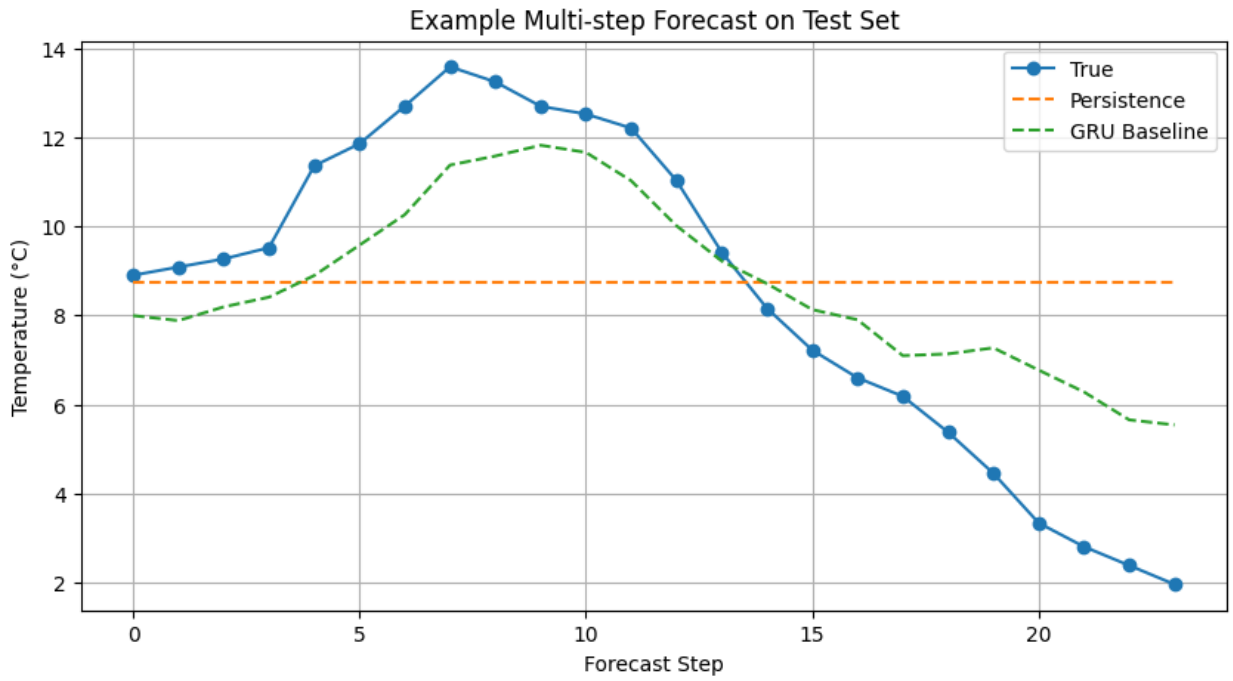


**Figure 1** - 24-hour forecast comparison: ground truth vs. persistence vs. GRU baseline (°C).

# 8. Synthetic Data Generation with TimeGAN

This section explores generative approaches for creating realistic synthetic weather sequences, with the goal of demonstrating data augmentation feasibility without information leakage.

## 8.1 Motivation and Experimental Role

The purpose of the TimeGAN component is to generate realistic synthetic multivariate weather sequences using only the training split, thereby avoiding any form of data leakage.

These synthetic sequences are intended for training-set augmentation, allowing us to assess whether synthetic data can improve forecasting performance, robustness, or generalization.

To remain consistent with the forecasting setup, the generative model is trained on multivariate sequences of length **LOOKBACK + HORIZON**, so that each generated sequence can later be divided into: - an input window of length **LOOKBACK** - a target forecasting horizon of length **HORIZON**

The impact of synthetic augmentation is then evaluated by comparing forecasting models trained under two conditions: 1. using real data only 2. using real data together with synthetic data

## 8.2 TimeGAN Training Data Preparation

Training data is converted into fixed-length sequences of LOOKBACK + HORIZON = 144 time steps, matching the forecasting window. This ensures generated sequences can be directly split into input–output pairs. Only the **training set** is used - validation and test data are never exposed to the generative model, preventing data leakage.

```
print("TimeGAN sequence length:", TIMEGAN_SEQ_LEN)
print("TimeGAN training sequences shape:", timegan_train_sequences.shape)
TimeGAN sequence length: 144
TimeGAN training sequences shape: (48885, 144, 16)
```

## 8.3 TimeGAN Model and Training Procedure

TimeGAN (Yoon et al., 2019) consists of five GRU-based sub-networks: **Embedder** (data → latent), **Recovery** (latent → data), **Generator** (noise → latent), **Supervisor** (temporal dynamics in latent space), and **Discriminator** (real vs. synthetic). Each uses 3 GRU layers with hidden dimension 24. Training follows a three-phase curriculum: autoencoder pretraining, supervisor pretraining, and adversarial training.

### *8.3.1 Autoencoder Pretraining*

**Phase 1:** The embedder and recovery networks are jointly trained to minimize reconstruction error (MSE) on real sequences. This stage is intended to learn a meaningful

latent representation of the multivariate weather dynamics before adversarial training begins.

The reconstruction loss decreases steadily from approximately **0.280** in the first epoch to **0.0023** by epoch 20, indicating that the autoencoder is able to compress and reconstruct the training sequences with high fidelity, see figure 2. This suggests that the latent space learned by the embedder is sufficiently informative to support the next stages of TimeGAN training.

```
autoencoder_history = timegan.pretrain_autoencoder(
    sequences=timegan_train_sequences,
    epochs=TIMEGAN_CONFIG["ae_epochs"],
    batch_size=TIMEGAN_CONFIG["batch_size"],
    verbose=1,
)
```

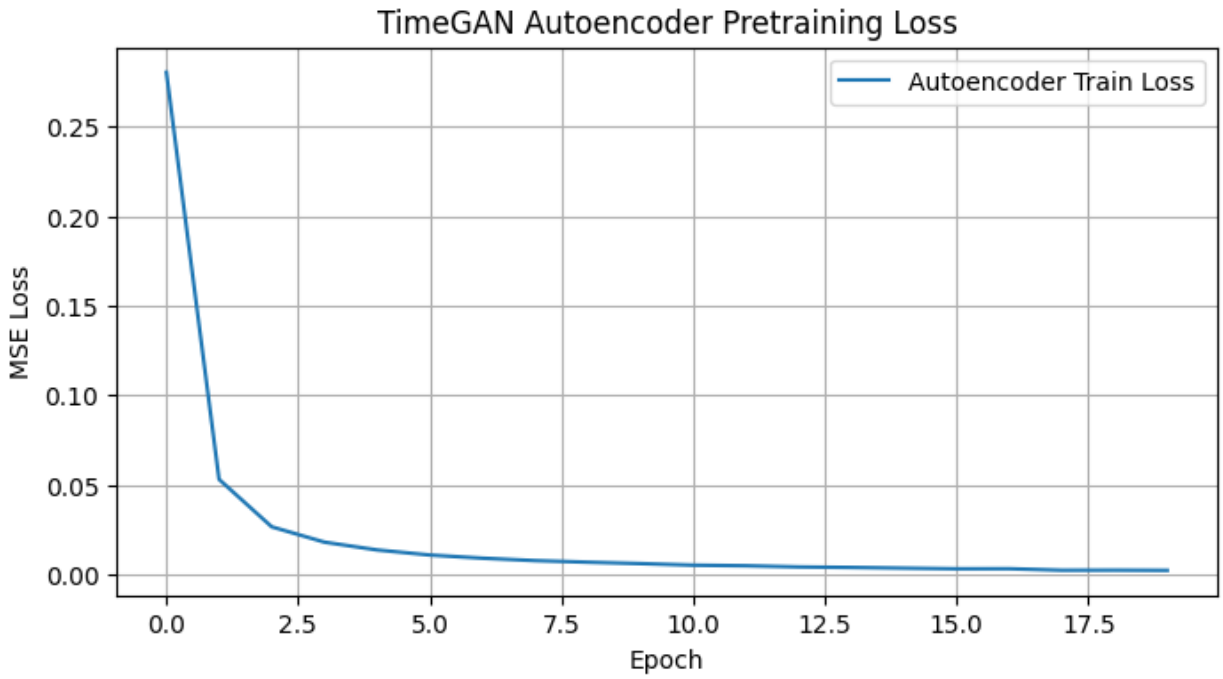


**Figure 2** - TimeGAN autoencoder pretraining loss (MSE) over 20 epochs.

### *8.3.2 Supervisor Pretraining*

**Phase 2:** The supervisor network is trained to predict the next latent time step from the current one, using latent representations produced by the embedder. This stage is designed to capture temporal dynamics in latent space before full adversarial optimization begins.

The supervisor loss decreases from approximately **0.0308** in the first epoch to **0.0076** by epoch 20, with the largest improvement occurring during the initial epochs and a gradual stabilization thereafter, see figure 3. This behaviour suggests that the latent temporal dynamics are being learned successfully, although convergence is less pronounced than

in the autoencoder stage. Such a pattern is expected, since temporal prediction in latent space is typically more challenging than direct reconstruction.

```
supervisor_history = timegan.pretrain_supervisor(
    sequences=timegan_train_sequences,
    epochs=TIMEGAN_CONFIG["sup_epochs"],
    batch_size=TIMEGAN_CONFIG["batch_size"],
    verbose=1,
)
```

### *8.3.3 Pretraining Loss Curves*

Combined visualization of autoencoder and supervisor pretraining losses to verify both networks converged before adversarial training.

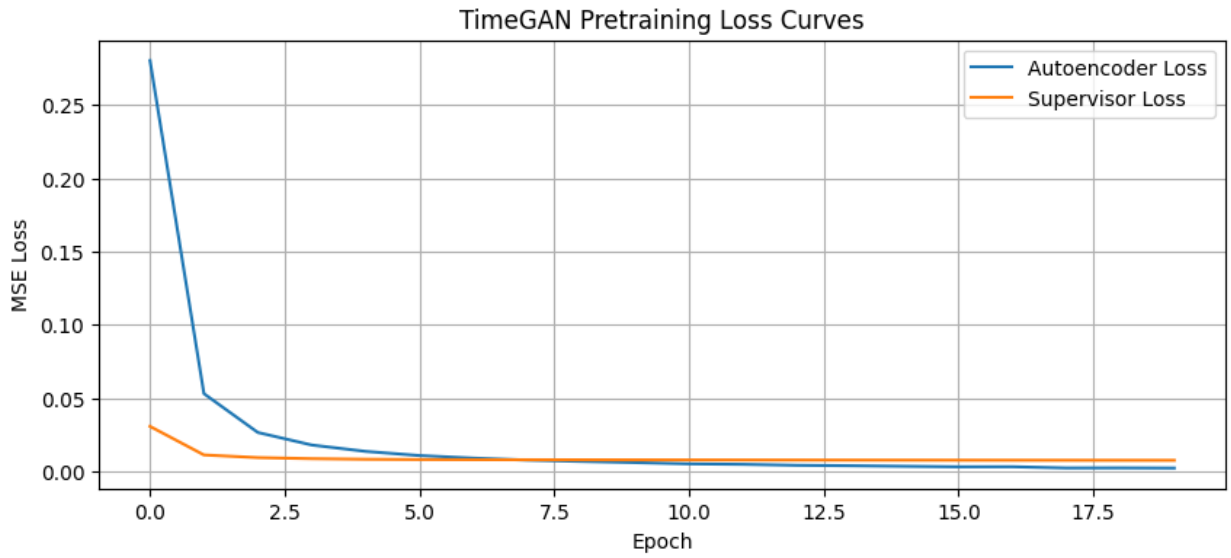


**Figure 3** - Combined pretraining loss curves for autoencoder and supervisor networks.

### *8.3.4 Adversarial Training*

**Phase 3:** The generator and discriminator are optimized adversarially. The total generator objective combines three components: an adversarial loss that encourages the generator to fool the discriminator, a supervised loss that promotes temporal coherence in latent space (weighted by **×100**), and a reconstruction loss that encourages fidelity in data space (weighted by **γ = 1.0**). A discriminator update threshold is used to reduce the risk of the discriminator becoming dominant too early in training.

During adversarial optimization, the discriminator loss remains within a moderate range, while the generator loss shows noticeable fluctuations across epochs. The supervised latent-dynamics component remains relatively stable and low, whereas the adversarial and reconstruction-related terms vary more substantially. This behaviour suggests that training remained numerically stable, but that the adversarial game did not converge as smoothly as the earlier pretraining stages.

The adversarial phase indicates that the model learned some generative structure without collapsing, but the loss trajectories also suggest that synthetic quality should be assessed carefully through downstream evaluation rather than inferred from adversarial losses alone.

```
adversarial_history = timegan.fit(
    sequences=timegan_train_sequences,
    epochs=TIMEGAN_CONFIG["adv_epochs"],
    batch_size=TIMEGAN_CONFIG["batch_size"],
    verbose=1,
)
```

### *8.3.5 Adversarial Training Loss Curves*

Figure 4, visualizes the evolution of discriminator and generator losses during the adversarial training phase. In principle, a well-behaved TimeGAN should exhibit a dynamic balance between generator and discriminator, without one network collapsing or overwhelmingly dominating the other.

In this experiment, the discriminator loss remains within a moderate range, while the generator total loss fluctuates more noticeably across epochs. The supervised component stays consistently low, suggesting that latent temporal coherence is being preserved, whereas the adversarial and reconstruction-related terms remain more variable. This pattern indicates that training remained stable enough to continue, but did not reach a clearly smooth adversarial equilibrium.

Therefore, the loss curves suggest partial learning of the generative objective, but they also reinforce the need to evaluate synthetic sequence quality directly before drawing conclusions about the usefulness of TimeGAN-based augmentation.

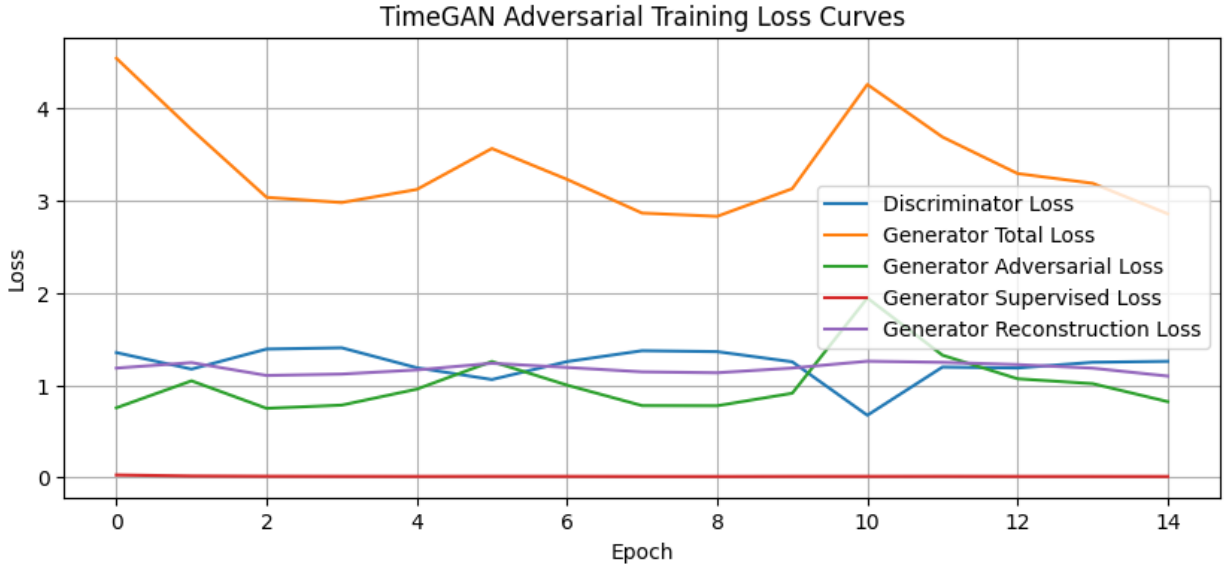


**Figure 4** - TimeGAN adversarial training loss curves (discriminator, generator, supervised, reconstruction).

## 8.4 Synthetic Sequence Quality Assessment

Before considering synthetic data for augmentation, we assess whether the TimeGAN produces sequences that are statistically and temporally faithful to the real training data through reconstruction checks and visual comparisons.

### *8.4.1 Autoencoder Reconstruction Check*

We pass real sequences through the embedder → recovery path to verify the latent space captures meaningful structure. Low reconstruction MSE indicates the autoencoder learned a faithful representation.

```
print("Reconstructed batch shape:", reconstructed_sequences.shape)
print("Reconstruction MSE on sample batch:", reconstruction_mse)
Reconstructed batch shape: (256, 144, 16)
Reconstruction MSE on sample batch: 0.0011650081324751588
```

### *8.4.2 Real vs Reconstructed Sequence*

Figure 5, compares a real temperature sequence with its reconstruction produced by the TimeGAN autoencoder. The two trajectories show a close overall overlap across most of the 144 time steps, indicating that the learned latent representation retains the dominant temporal structure of the original series.

The reconstruction captures both the broad trend and most of the short-term fluctuations, although small deviations remain at some local peaks, turning points, and sharp declines. This behaviour is consistent with the low reconstruction loss observed during pretraining and suggests that the embedder–recovery pair is able to encode the input sequences with high fidelity. However, strong reconstruction quality alone does not guarantee high-quality synthetic generation, so the usefulness of TimeGAN must still be assessed through the generated samples and the downstream augmentation experiment.

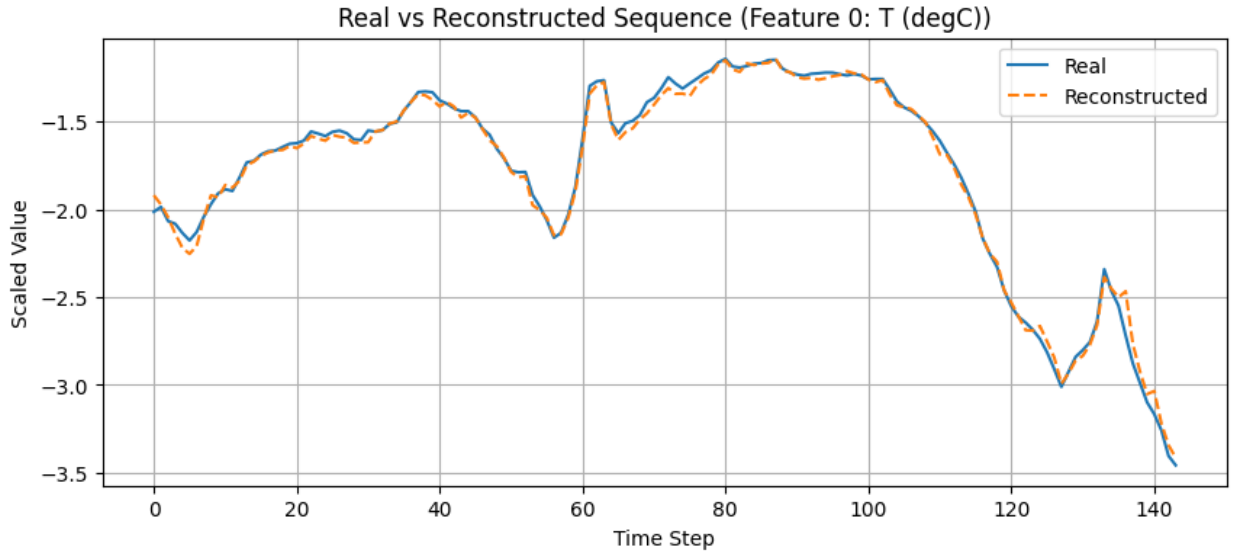


**Figure 5** - Real vs. reconstructed temperature sequence through the TimeGAN autoencoder.

### *8.4.3 Synthetic Sequence Preview*

A small batch of synthetic sequences is generated as an initial sanity check on the output of the trained generator. The generated batch has shape **(8, 144, 16)**, which matches the expected sequence length and feature dimensionality used throughout the forecasting pipeline.

The observed value range, from approximately **-1.003** to **0.776** in scaled space, indicates that the generator is producing bounded numerical outputs without obvious explosion or instability. However, this range is noticeably narrower than the variability typically observed in the real standardized data, suggesting that the synthetic sequences may still underrepresent part of the natural amplitude of the original series.

Therefore, this preview should be interpreted only as a basic sanity check. The actual usefulness of the generated sequences must be assessed through direct visual inspection and, more importantly, through the downstream comparison between forecasting models trained with and without synthetic augmentation.

```
synthetic_sequences_preview = timegan.generate(8)

print("Synthetic preview shape:", synthetic_sequences_preview.shape)
print("Synthetic preview min:", synthetic_sequences_preview.min())
print("Synthetic preview max:", synthetic_sequences_preview.max())
Synthetic preview shape: (8, 144, 16)
Synthetic preview min: -1.002639
Synthetic preview max: 0.7755798
```

### *8.4.4 Real vs Synthetic Sequence*

Figure 6, compares a real training sequence with a fully generated synthetic sequence for the temperature feature. This visualization provides a qualitative assessment of whether the generator is able to reproduce realistic temporal dynamics and value distributions after adversarial training.

Although the autoencoder reconstruction stage achieved high fidelity, the fully generated synthetic sequence does not adequately match the structure of the real signal. In particular, the synthetic trajectory shows much lower variability, reduced amplitude, and a flatter temporal pattern, failing to reproduce the richer dynamics observed in the real temperature series. This indicates that, despite successful latent reconstruction, the adversarial generation stage did not learn a sufficiently realistic data distribution.

The synthetic quality assessment suggests that the generated sequences are not yet reliable enough for data augmentation. While the autoencoder reconstruction is strong (with reconstruction MSE around **0.0012**), the synthetic samples themselves do not sufficiently replicate the temporal behaviour of the original data. For this reason, adding synthetic data to the forecasting pipeline would risk introducing noise and degrading predictive performance. Therefore, the TimeGAN component is retained as a proof-of-

concept exploration of generative modelling, but synthetic augmentation was not used in the final forecasting models.

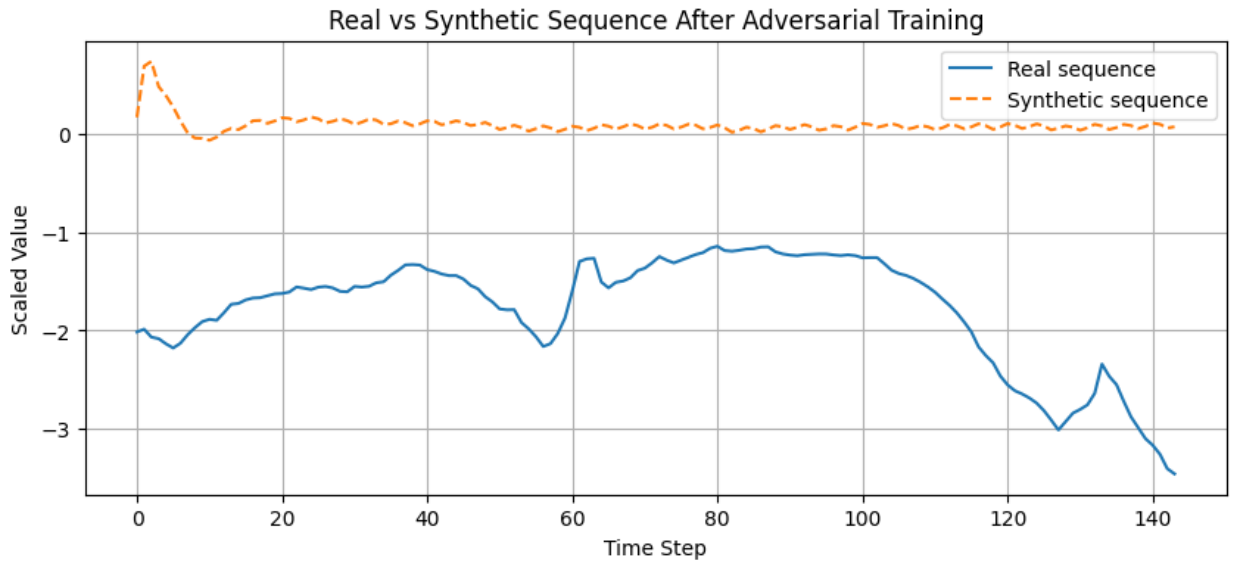


**Figure 6** - Real training sequence vs. TimeGAN-generated synthetic sequence (temperature feature).

# 9. Evolutionary Optimization of the Forecasting Pipeline

## 9.1 Motivation and Search Strategy

The evolutionary optimization stage aims to improve the forecasting pipeline beyond the fixed GRU baseline by systematically exploring alternative architectural, preprocessing, training, and windowing configurations.

Instead of relying on a single manually selected model or Bayesian optimization from previous work, this stage uses a genetic search strategy based on:

- population initialization
- tournament selection
- crossover
- mutation
- elitism

Each candidate solution represents a full forecasting pipeline configuration, including GRU architecture, optimizer, loss function, regularization, batch size, scaling method, and input lookback length. The fitness of each candidate is evaluated on the **validation set only**, using forecasting performance in the original temperature scale, so that evolutionary selection remains aligned with the final project objective and avoids test-set leakage.

## 9.2 Search Space Definition

The evolutionary search space spans **17 optimization dimensions** grouped into five main categories:

- **Architecture:** number of GRU layers (1–2), hidden units per layer (units1: 64–128, units2: 64–128, units3: 32–96), optional dense layer (0–256 units), and dense activation function (ReLU, GELU, ELU, LeakyReLU)
- **Regularization and optimization control:** dropout (0.0–0.3), L2 regularization (0 to 10^{-4}), Gaussian noise (0.0 or 0.01), and gradient clipping (0.5–5.0)
- **Training:** optimizer (Adam, AdamW), weight decay (0 to 10^{-4}), learning rate (10^{-4} to 10^{-3}), loss function (MSE, MAE, Huber δ=1, Huber δ=2), and batch size (128 or 256)
- **Preprocessing:** scaler type (standard, robust, minmax)
- **Windowing:** input lookback length (96, 120, 144 hours)

The final search space was deliberately narrowed relative to earlier exploratory versions in order to keep the evolutionary budget computationally tractable while still allowing meaningful variation in architecture, preprocessing, training dynamics, and temporal context length. In particular, the search was restricted to 1- and 2-layer GRU models and moderate hidden sizes, focusing computation on configurations more likely to be competitive for this dataset.

### *9.2.1 Windowing Search Extension*

Unlike the mini-project, the windowing configuration was not treated as fixed. In addition to architecture and training hyperparameters, the evolutionary search was extended to explore alternative lookback lengths while keeping the 24-hour forecast horizon fixed. This allowed the optimization process to jointly search over the forecasting model and the temporal context length used as input.

## 9.3 Evolutionary Representation and Constraints

Each evolutionary individual is represented as a **genotype**, implemented as a dictionary that maps gene names to admissible values from the predefined search space. This genotype encodes a full forecasting pipeline configuration, including architectural choices, optimization settings, preprocessing strategy, and input lookback length.

To ensure that sampled and evolved solutions remain valid and computationally meaningful, a set of constraints is enforced throughout the search process. In particular: - GRU layer widths are constrained to be monotonically non-increasing (units2 ≤ units1, units3 ≤ units2), preventing unnecessarily inconsistent architectures - weight_decay is forced to 0 when the selected optimizer is standard **Adam**, since explicit weight decay is only meaningful for **AdamW** - all constraints are re-applied after initialization, crossover, and mutation, ensuring that every evaluated genotype corresponds to a valid forecasting configuration

This constrained representation helps reduce unproductive regions of the search space and improves the efficiency and fairness of the evolutionary search.

```
from src.evolution.genotype import sample_genotype

example_genotype = sample_genotype()
example_genotype
{'n_layers': 2,
 'units1': 128,
 'units2': 64,
 'units3': 64,
 'dropout': 0.0,
 'l2': 0.0,
 'dense_units': 64,
 'dense_activation': 'elu',
 'learning_rate': 0.001,
 'batch_size': 128,
 'clipnorm': 0.5,
 'optimizer_name': 'adamw',
 'weight_decay': 1e-05,
 'loss_name': 'huber2',
 'gaussian_noise_std': 0.01,
```

```
  'scaler_name': 'standard',
  'lookback': 120}
```

## 9.4 Fitness Function

The fitness function encapsulates the full forecasting pipeline within a single evaluation procedure, from preprocessing and window construction to model training and validation forecasting. Each candidate genotype is decoded into a concrete forecasting configuration, trained on the training split, and evaluated exclusively on the validation split.

For each candidate configuration, the evaluation pipeline performs the following steps:

1. feature scaling using the candidate preprocessing strategy
2. supervised window generation using the candidate lookback length
3. GRU model construction according to the candidate architectural and training genes
4. model training on the training split
5. prediction on the validation split
6. computation of validation forecasting error

The evolutionary fitness is defined as the **validation MAE in the original temperature scale (°C)**. Scaled metrics are also recorded for analysis, but the optimization objective is to minimize validation error in physically interpretable units. This makes the evolutionary search directly aligned with the final forecasting objective while keeping the test set completely untouched.

## 9.5 Evolutionary Search Execution

The genetic algorithm was executed with a **population size of 20** over **15 generations**, resulting in **300 individual evaluations**. The search used **tournament selection** (k = 3), **uniform crossover**, **per-gene mutation** (rate = 0.2), and **elitism**, ensuring that the best solutions were preserved across generations.

Each candidate was trained for up to **20 epochs**, with early stopping and learning-rate reduction used to improve training efficiency and stability.

The evolutionary fitness was defined as **validation MAE in the original temperature scale (°C)**. This choice ensures that candidate solutions are compared in physically interpretable units and that the evaluation remains invariant to the selected scaler.

The best validation fitness improved over the first generations and then stabilized, indicating that the search was able to identify competitive regions of the configuration space without showing uncontrolled divergence.

The best individual found in this extended search achieved a validation MAE of approximately **1.636 °C** and corresponded to the following configuration:

- n_layers = 2
- units1 = 64
- units2 = 64
- units3 = 32
- dropout = 0.0
- l2 = 1e-5
- dense_units = 256
- dense_activation = relu
- learning_rate = 3e-4
- batch_size = 256
- clipnorm = 5.0
- optimizer_name = adamw
- weight_decay = 0.0
- loss_name = mae
- gaussian_noise_std = 0.0
- scaler_name = robust
- lookback = 144

This result is particularly relevant because it confirms that **windowing strategy was effectively included in the optimization process**: the best candidate selected a lookback of **144 hours**, rather than the default 120-hour setting. Therefore, the forecasting window was not treated as fixed, in line with the project requirements.

**Overfitting mitigation:** fitness was computed on the **validation set only**, while the **test set remained untouched** throughout the evolutionary search. In addition, robustness was later assessed by re-evaluating selected configurations across multiple random seeds (see Section 10.6).

## 9.6 Best Configuration and Retraining

The best configuration found by the evolutionary search is defined as the individual with the **lowest validation MAE** across all generations. This best genotype encodes a complete forecasting pipeline, including preprocessing strategy, GRU architecture, regularization choices, optimizer settings, loss function, batch size, and input lookback length.

In the extended evolutionary search that explicitly included windowing optimization, the best individual selected the following configuration:

- n_layers = 2

- units1 = 64
- units2 = 64
- units3 = 32
- dropout = 0.0
- l2 = 1e-5
- dense_units = 256
- dense_activation = relu
- learning_rate = 3e-4
- batch_size = 256
- clipnorm = 5.0
- optimizer_name = adamw
- weight_decay = 0.0
- loss_name = mae
- gaussian_noise_std = 0.0
- scaler_name = robust
- lookback = 144

This result confirms that the evolutionary process was able to optimize not only model and training hyperparameters, but also the temporal context used as model input.

The selected configuration is therefore retained as the best candidate emerging from the windowing-aware evolutionary search and is used for retraining and subsequent comparison with the other final forecasting pipelines.

## 9.7 Top Candidate Configurations

We display the top-ranked individuals from the final EA generation to understand the diversity of solutions discovered. This helps assess whether the EA converged to a single region of the search space or explored multiple competitive alternatives.

**Table 8** - Top candidate configurations discovered across all EA generations.

| Parameter | Candidate 1 | Candidate 2 | Candidate 3 | Candidate 4 | Candidate 5 |
|---|---|---|---|---|---|
| **n_layers** | 2 | 2 | 2 | 2 | 2 |
| **units1/units2/units3** | 64 / 64 / 32 | 96 / 96 / 64 | 128 / 128 / 32 | 64 / 64 / 32 | 128 / 64 / 64 |
| **dropout** | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 |
| **L2** | 0.000010 | 0.000001 | 0.000100 | 0.000001 | 0.000001 |
| **dense_units** | 256 | 256 | 256 | 256 | 256 |
| **dense_activation** | relu | relu | relu | relu | relu |
| **learning_rate** | 0.0003 | 0.0003 | 0.0003 | 0.0003 | 0.0002 |
| **batch_size** | 256 | 256 | 256 | 256 | 128 |
| **clipnorm** | 5.0 | 5.0 | 5.0 | 5.0 | 5.0 |
| **optimizer_name** | adamw | adam | adam | adam | adam |
| **weight_decay** | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 |
| **loss_name** | mae | mae | mae | mae | mae |
| **gaussian_noise_std** | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 |
| **scaler_name** | robust | robust | robust | robust | robust |
| **lookback** | 144 | 144 | 144 | 144 | 144 |
| fitness_mae_degC | 1.635730 | 1.635795 | 1.636511 | 1.637370 | 1.638159 |
| rmse_degC | 2.237388 | 2.229955 | 2.242214 | 2.239683 | 2.237043 |
| mae_scaled | 0.132586 | 0.132592 | 0.132650 | 0.132719 | 0.132783 |
| rmse_scaled | 0.181355 | 0.180752 | 0.181746 | 0.181541 | 0.181327 |

# 10. Retraining and Final Model Selection

The best evolutionary configuration was identified under a limited training budget during the search stage. To obtain a fair final comparison, the main candidate models are retrained with an expanded training budget, evaluated on the held-out test set, and further analysed in terms of robustness, explainability, and efficiency.

## 10.1 Selection of Final Candidates

This section defines the final forecasting candidates to be compared under a common retraining and evaluation protocol. Two main models are selected at this stage:

1. the **GRU Baseline Official**, inherited from the best-performing configuration of the previous mini-project and used as the main deep learning reference;
2. the **Best Evolutionary GRU**, corresponding to the best configuration found by the extended evolutionary search, including preprocessing, architecture, training hyperparameters, and lookback optimization.

These two candidates are retrained under a larger training budget and evaluated on the held-out test set. Their results are then used as the basis for the final model comparison, which is later extended through explainability-guided pruning and robustness analysis.

**Table 9** - Final candidate configurations: GRU baseline vs. best EA-discovered pipeline.

| Parameter | GRU Baseline Official | Best Evolutionary GRU |
|---|---|---|
| **n_layers** | 2 | 2 |
| **units1** | 96 | 64 |
| **units2** | 64 | 64 |
| **units3** | 96 | 32 |
| **dropout** | 0.0 | 0.0 |
| **l2** | 0.000001 | 0.000010 |
| **dense_units** | 256 | 256 |
| **dense_activation** | relu | relu |
| **learning_rate** | 0.0002 | 0.0003 |
| **clipnorm** | 2.0 | 5.0 |
| **optimizer_name** | adamw | adamw |
| **weight_decay** | 0.000001 | 0.000000 |
| **loss_name** | huber1 | mae |
| **gaussian_noise_std** | 0.0 | 0.0 |
| **batch_size** | 128 | 256 |
| **scaler_name** | standard | robust |
| **lookback** | 120 | 144.0 |

## 10.2 Retraining Procedure

Both candidate models are retrained from scratch under a larger training budget (50 epochs) than the one used during evolutionary search, using their respective preprocessing and windowing configurations. The **GRU Baseline Official** uses **StandardScaler** with the default lookback setting, while the **Best Evolutionary GRU** uses the scaler and lookback length selected by the evolutionary search, namely **RobustScaler** and a **144-hour lookback**.

To ensure a fair comparison, the same training protocol is applied to both models, including early stopping and learning-rate reduction on plateau. This retraining stage provides a more reliable estimate of each candidate's final forecasting capability before test-set evaluation.

We define a helper function to prepare scaled and windowed data for any pipeline configuration, supporting dynamic lookback and scaler type.

```
def prepare_data_for_cfg(cfg, df_train, df_val, df_test, final_feature_cols, targ
et_idx, lookback, horizon):
    scaler = get_scaler(cfg["scaler_name"])

    X_train_df = df_train[final_feature_cols].copy()
    X_val_df = df_val[final_feature_cols].copy()
    X_test_df = df_test[final_feature_cols].copy()

    X_train_scaled = scaler.fit_transform(X_train_df)
    X_val_scaled = scaler.transform(X_val_df)
    X_test_scaled = scaler.transform(X_test_df)

    X_train, y_train = make_windows(X_train_scaled, target_idx, lookback, horizo
n)
    X_val, y_val = make_windows(X_val_scaled, target_idx, lookback, horizon)
    X_test, y_test = make_windows(X_test_scaled, target_idx, lookback, horizon)

    return scaler, X_train, y_train, X_val, y_val, X_test, y_test
```

We define a helper function to prepare scaled and windowed data for any pipeline configuration, supporting dynamic lookback and scaler type. Then, we define a helper function that prepares data (scaling + windowing) for any configuration, handling dynamic lookback and scaler type, and prepare data for both candidates using their respective scalers and lookback windows.

Both final candidates are rebuilt from scratch and retrained under a larger training budget than the one used during evolutionary search, with a maximum of **50 epochs** and identical callback conditions. In practice, training may stop earlier due to **early stopping**, as validation performance is monitored throughout optimization.

The **GRU Baseline Official** uses **StandardScaler** with the default lookback configuration, while the **Best Evolutionary GRU** uses the preprocessing and lookback settings selected by the evolutionary search, namely **RobustScaler** and a **144-hour lookback**. This retraining stage provides a fairer and more reliable comparison before final test-set evaluation. The final GRU baseline was retrained from scratch under the same callback configuration used throughout the project, with a maximum budget of 50 epochs. In practice, training converged earlier due to early stopping. The best validation loss was **0.034551** at **epoch 14**, while the best validation MAE reached **0.194307** at **epoch 20**. This indicates stable convergence and confirms that the baseline model did not require the full nominal epoch budget to reach its best validation performance, see figure 7.

```
Best val_loss: 0.034551 (epoch 14)
Best val_mae:  0.194307 (epoch 20)
```

Report upon the best validation metrics achieved during evolutionary model training and the best validation loss and MAE achieved during evolutionary model training:

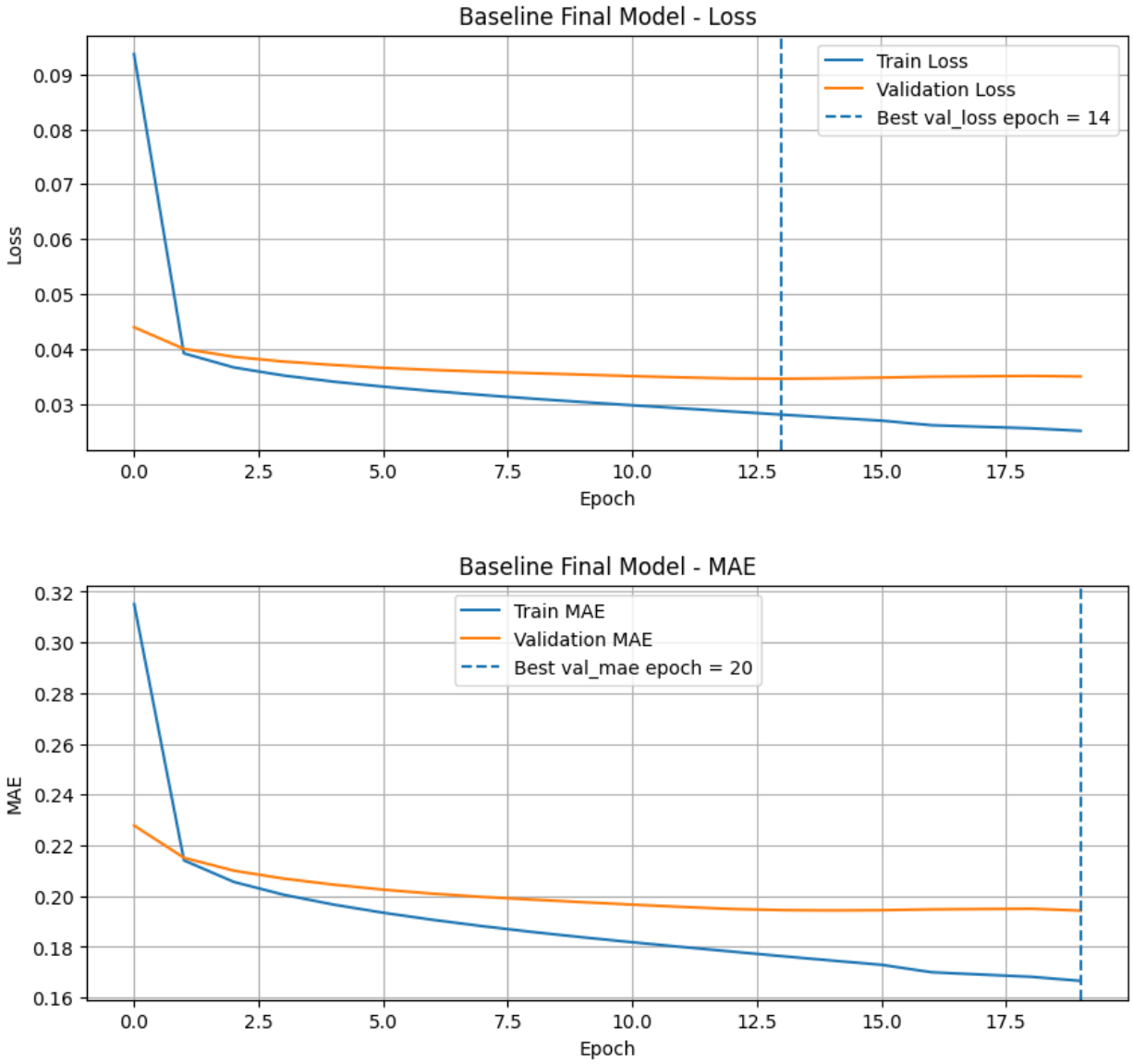


**Figure 7** - Baseline GRU training and validation loss curves with best epoch marker.

The evolutionary model is retrained below using the preprocessing and windowing configuration selected by the evolutionary search, namely **RobustScaler** and a **144-hour lookback window**. Unlike the baseline retraining, this model continues to improve steadily across the 50-epoch budget, indicating that the optimized configuration remains trainable and competitive under a larger retraining schedule. The final evolutionary GRU was retrained using the preprocessing and windowing configuration selected by the evolutionary search, namely **RobustScaler** and a **144-hour lookback**. In contrast to the baseline model, whose validation performance stabilized earlier, the evolutionary configuration continued to improve for a longer period during retraining. The best validation loss and validation MAE were both achieved at **epoch 43**, reaching **0.137592** and **0.135395**, respectively, see figure 8. This indicates a stable and gradual optimization process under the expanded retraining budget, suggesting that the evolutionary configuration benefits from a longer training schedule than the manually defined baseline.

```
Best val_loss: 0.137592 (epoch 43)
Best val_mae:  0.135395 (epoch 43)
```

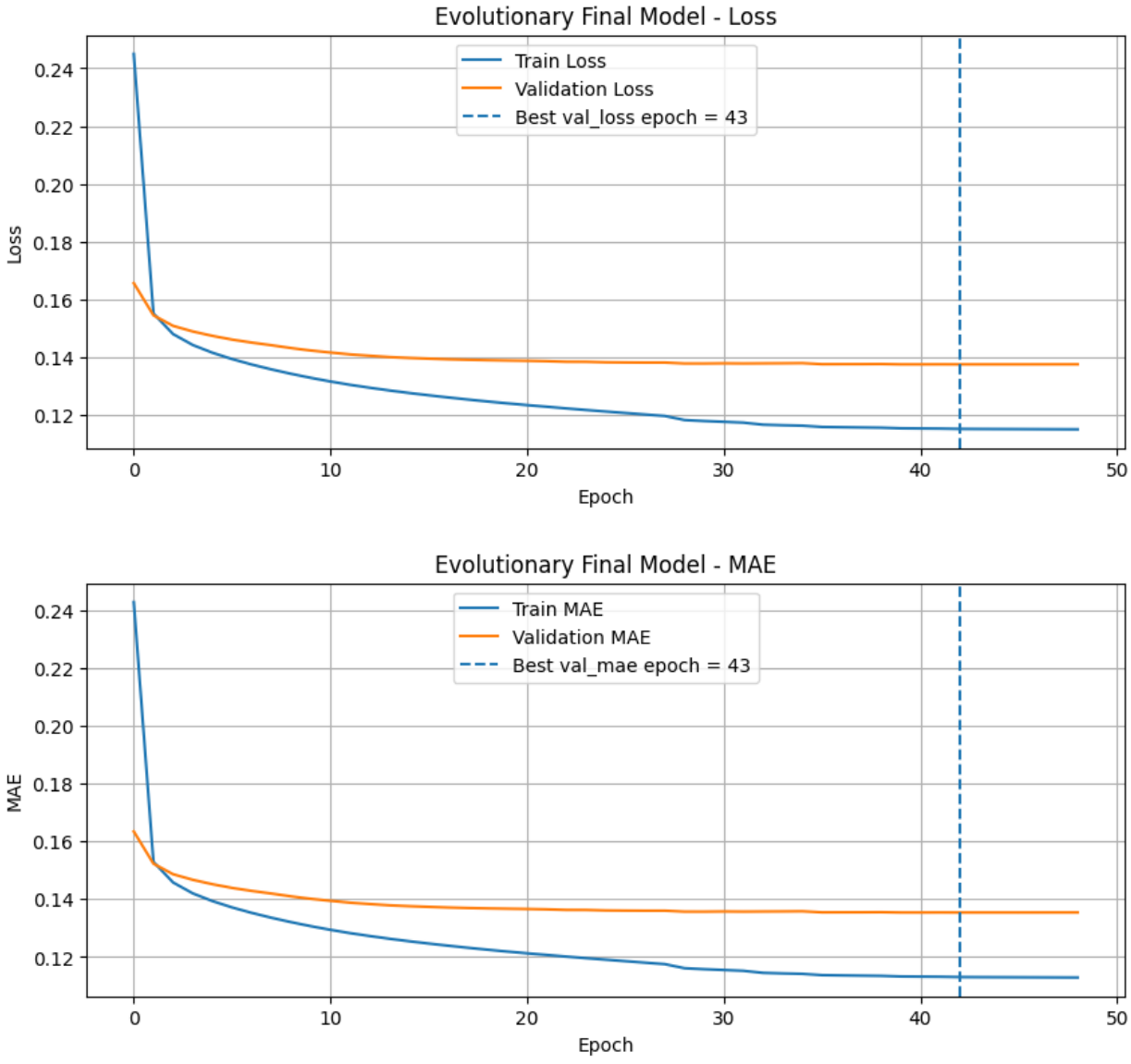


**Figure 8** - EA-optimized GRU training and validation loss curves with best epoch marker.

## 10.3 Final Test Set Evaluation

Both retrained models are evaluated on the held-out test set, which has been completely untouched throughout the optimization process. We report metrics in both scaled and original (°C) space.

### *10.3.1 Test Predictions*

Generate predictions for both models on the test set.

```
print("Baseline test prediction shape:", y_pred_baseline_test.shape)
print("Evolutionary test prediction shape:", y_pred_evo_test.shape)
Baseline test prediction shape: (10364, 24)
Evolutionary test prediction shape: (10364, 24)
```

### *10.3.2 Test Metrics in Scaled Space*

After retraining, both final candidate models were evaluated on the held-out test set. Table 10.3 first reports the results in **scaled space**, which allows a direct comparison of predictive accuracy under each model's own preprocessing regime.

The **GRU Baseline Official** achieved a test **MAE of 0.1909** and **RMSE of 0.2537** in scaled space. In contrast, the **Best Evolutionary GRU** achieved a substantially lower test **MAE of 0.1295** and **RMSE of 0.1748**. This indicates a clear improvement in forecasting performance after evolutionary optimization.

These results show that the optimized pipeline generalizes better than the manually defined baseline, even before converting the predictions back to the original temperature scale.

```
print("Baseline Test MAE (scaled):", baseline_test_scaled_metrics["mae_scaled"])
print("Baseline Test RMSE (scaled):", baseline_test_scaled_metrics["rmse_scaled
"])
print("Evolutionary Test MAE (scaled):", evo_test_scaled_metrics["mae_scaled"])
print("Evolutionary Test RMSE (scaled):", evo_test_scaled_metrics["rmse_scaled"])
Baseline Test MAE (scaled): 0.1908517425435448
Baseline Test RMSE (scaled): 0.2536655440317924
Evolutionary Test MAE (scaled): 0.12950307188826218
Evolutionary Test RMSE (scaled): 0.17483887194295933
```

### *10.3.3 Test Metrics in Original Temperature Scale*

MAE and RMSE in degrees Celsius - the primary evaluation metrics with direct physical interpretation. These are obtained by inverse-transforming predictions using each model's respective scaler statistics.

To obtain physically interpretable results, both predictions and targets were inverse-transformed back to degrees Celsius, since the two models use different scalers (StandardScaler vs RobustScaler), each inverse transformation uses its own fitted parameters to ensure correctness.. The final comparison confirms that the evolutionary optimization improved the forecasting system not only in scaled space, but also in the original temperature domain.

The **GRU Baseline Official** achieved a test **MAE of 1.650 °C** and **RMSE of 2.193 °C**. The **Best Evolutionary GRU** improved these results to a test **MAE of 1.598 °C** and **RMSE of 2.157 °C**. Therefore, the evolutionary model outperformed the baseline on both error metrics under the final held-out test evaluation.

```
print("Baseline Test MAE (°C):", baseline_test_original_metrics["mae"])
print("Baseline Test RMSE (°C):", baseline_test_original_metrics["rmse"])
print("Evolutionary Test MAE (°C):", evo_test_original_metrics["mae"])
print("Evolutionary Test RMSE (°C):", evo_test_original_metrics["rmse"])
Baseline Test MAE (°C): 1.649835827188564
Baseline Test RMSE (°C): 2.1928356382255343
Evolutionary Test MAE (°C): 1.597690189808148
Evolutionary Test RMSE (°C): 2.1570017330662847
```

Inverse-scale predictions and ground truth back to degrees Celsius using each model's respective scaler.

## 10.4 Final Model Comparison

The final comparison shows that the **Best Evolutionary GRU** outperformed the **GRU Baseline Official** across all reported metrics.

In scaled space, the evolutionary model reduced test MAE from **0.1909** to **0.1295** and test RMSE from **0.2537** to **0.1748**. After inverse transformation to the original temperature scale, the improvement remained consistent: test MAE decreased from **1.650 °C** to **1.598 °C**, and test RMSE decreased from **2.193 °C** to **2.157 °C**.

These results confirm that the evolutionary optimization produced a stronger forecasting pipeline than the manually defined baseline, with consistent gains in both normalized and physically interpretable evaluation metrics, see table 10.

**Table 10** - Final model comparison on the held-out test set (scaled and °C metrics).

| | Model | MAE_scaled | RMSE_scaled | MAE_degC | RMSE_degC |
|---|---|---|---|---|---|
| 0 | GRU Baseline Official | 0.190852 | 0.253666 | 1.649836 | 2.192836 |
| 1 | Best Evolutionary GRU | 0.129503 | 0.174839 | 1.597690 | 2.157002 |

## 10.5 Final Model Selection

The final comparison between the official GRU baseline and the best evolutionary GRU shows that the evolutionary optimization process was able to find a configuration that is both more accurate and more compact.

The official GRU baseline achieved a test MAE of 1.650°C and an RMSE of 2.193°C (86,744 parameters), while the best evolutionary GRU achieved a lower MAE of 1.598°C and RMSE of 2.157°C with only 63,512 parameters - a 3.2% MAE improvement using 27% fewer parameters.

The evolutionary model demonstrates that the search process was capable of refining the forecasting configuration beyond the manually fixed baseline. Therefore, the best evolutionary GRU is selected as the final forecasting model for subsequent XAI and efficiency analysis.

## 10.6 Robustness Across Random Seeds

Since evolutionary search may overfit to the validation set through a single seed, we evaluate the best EA configuration across 3 different random seeds (7, 21, 42). This provides a mean ± std estimate of performance, validating that the discovered configuration generalizes beyond a single training initialization.

To reduce the risk of validation-set overfitting during evolutionary selection, the best evolutionary configuration was re-evaluated across multiple random seeds while keeping the test set untouched.

The results remained consistent across the three tested seeds. The model achieved test MAE values between **1.579 °C** and **1.610 °C**, and test RMSE values between **2.129 °C** and **2.175 °C**. The mean performance across seeds was approximately **1.593 °C MAE** and **2.151 °C RMSE**, indicating that the selected evolutionary configuration is reasonably stable and does not rely on a single favourable random initialization, see table 11.

**Table 11** - EA-optimized model performance across 3 random seeds (7, 21, 42).

| | seed | lookback | mae_degC | rmse_degC |
|---|---|---|---|---|
| 0 | 7 | 144 | 1.610165 | 2.174758 |
| 1 | 21 | 144 | 1.578997 | 2.129135 |
| 2 | 42 | 144 | 1.589364 | 2.149213 |

# 11. Explainable AI

This section investigates the behaviour of the optimized forecasting pipeline through Explainable AI (XAI) techniques. The analysis combines **permutation importance** for global feature relevance, **gradient-based saliency** for local input attribution, and an **XAI-guided feature pruning** experiment designed to test whether less important variables can be removed without harming predictive performance.

## 11.1 Global Explainability

Global explainability is assessed through **permutation importance**, which measures the contribution of each input feature by randomly shuffling its values across samples and observing the corresponding increase in prediction error. A larger increase in MAE indicates that the forecasting model relies more strongly on that feature. Applied to the **Best Evolutionary GRU** on the test set, permutation importance shows that **past temperature (T (degC))** is by far the most influential variable, followed by cyclical temporal covariates such as **doy_cos**, **hour_cos**, and **hour_sin**. Among the meteorological covariates, **maximum wind speed**, **relative humidity**, **wind-vector components**, and **pressure** also contribute meaningfully, while features such as **gust_ratio**, **doy_sin**, and the raw wind-direction encoding have comparatively limited influence, see table 12 and figure 9. This ranking is consistent with meteorological intuition: recent temperature history is the dominant predictor, while daily and seasonal periodicity provide important contextual structure for forecasting.

**Table 12** - Permutation feature importance ranking (mean MAE increase when feature is shuffled).

| | feature | importance_mean | importance_std |
|---|---|---|---|
| 0 | T (degC) | 0.481514 | 0.001128 |
| 9 | doy_cos | 0.072117 | 0.000336 |
| 7 | hour_cos | 0.061358 | 0.000557 |
| 6 | hour_sin | 0.057591 | 0.000525 |
| 4 | max. wv (m/s) | 0.035411 | 0.000663 |
| 2 | rh (%) | 0.028640 | 0.000586 |
| 12 | wx | 0.025296 | 0.000444 |
| 3 | wv (m/s) | 0.025189 | 0.000053 |
| 1 | p (mbar) | 0.017491 | 0.000227 |
| 14 | wind_gap | 0.016722 | 0.000360 |
| 11 | wd_cos | 0.012944 | 0.000493 |
| 13 | wy | 0.011039 | 0.000166 |
| 10 | wd_sin | 0.007232 | 0.000151 |
| 5 | wd (deg) | 0.006559 | 0.000106 |
| 15 | gust_ratio | 0.003199 | 0.000046 |
| 8 | doy_sin | 0.002842 | 0.000284 |

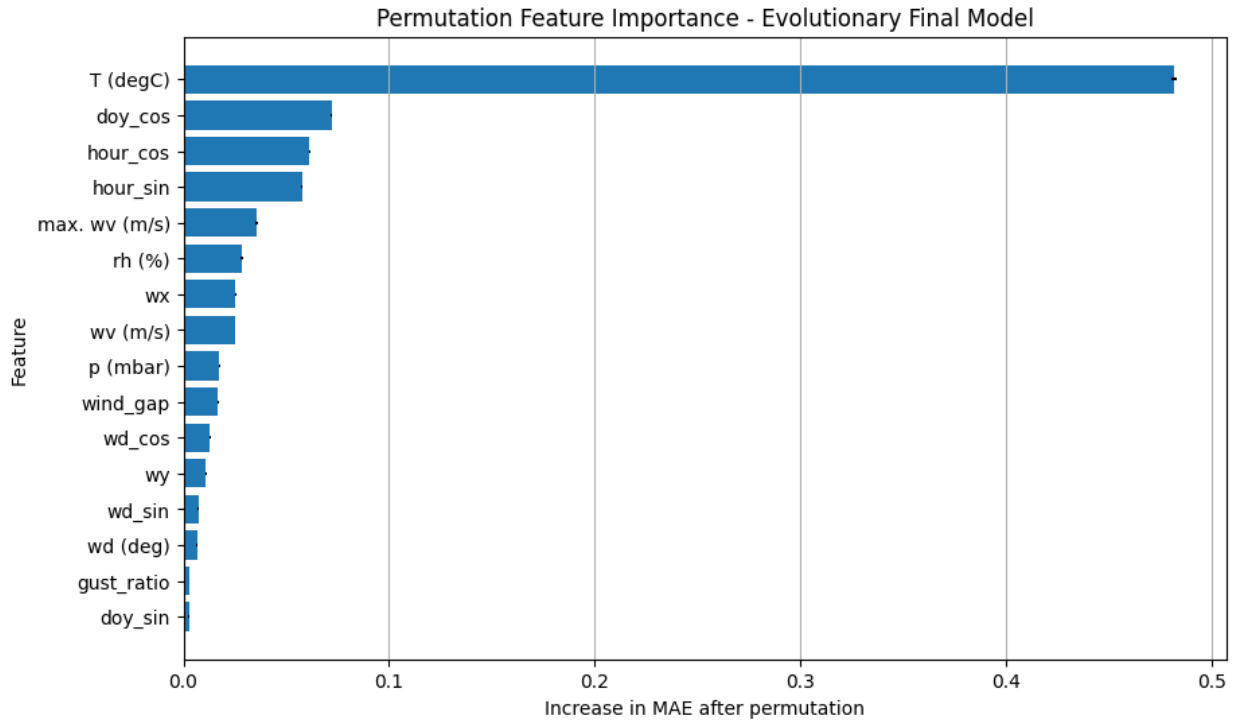


**Figure 9** - Permutation feature importance: MAE increase per feature (higher = more important).

## 11.2 Local Explainability

For local explanation, we compute the gradient of the model's output with respect to the input for specific samples. The absolute gradient magnitude indicates which time steps and features most influence the prediction at that point, providing a per-sample attribution map.

### *11.2.1 Feature-Level Local Importance*

Local explainability is analysed through **gradient-based saliency**, aggregated across all time steps for each input feature in a single test forecast. This produces a feature-level importance ranking that reflects which variables most influenced the model's prediction for that specific sample.

For the analysed example, **past temperature (T (degC))** is again the dominant feature, followed by temporal covariates such as **hour_cos**, **doy_cos**, and **hour_sin**, as well as **atmospheric pressure**. This indicates that, for this particular forecast, the model relied primarily on recent thermal history together with strong daily and seasonal temporal context. Secondary contributions came from **maximum wind speed**, **relative humidity**, and selected wind-derived components, while raw wind direction and gust_ratio had comparatively little local influence see table 13 and figure 10.

This local ranking is broadly consistent with the global permutation importance results, which strengthens the interpretation that the model combines recent temperature, temporal periodicity, and selected atmospheric variables when producing its forecasts.

**Table 13** - Local feature-level gradient saliency for sample 0 (aggregated across time steps).

| | feature | saliency |
|---|---|---|
| 0 | T (degC) | 0.007824 |
| 7 | hour_cos | 0.003603 |
| 1 | p (mbar) | 0.002538 |
| 9 | doy_cos | 0.002444 |
| 6 | hour_sin | 0.002331 |
| 4 | max. wv (m/s) | 0.001505 |
| 2 | rh (%) | 0.001111 |
| 13 | wy | 0.001029 |
| 8 | doy_sin | 0.000939 |
| 3 | wv (m/s) | 0.000931 |
| 11 | wd_cos | 0.000782 |
| 14 | wind_gap | 0.000728 |
| 12 | wx | 0.000713 |
| 10 | wd_sin | 0.000604 |
| 15 | gust_ratio | 0.000461 |
| 5 | wd (deg) | 0.000382 |

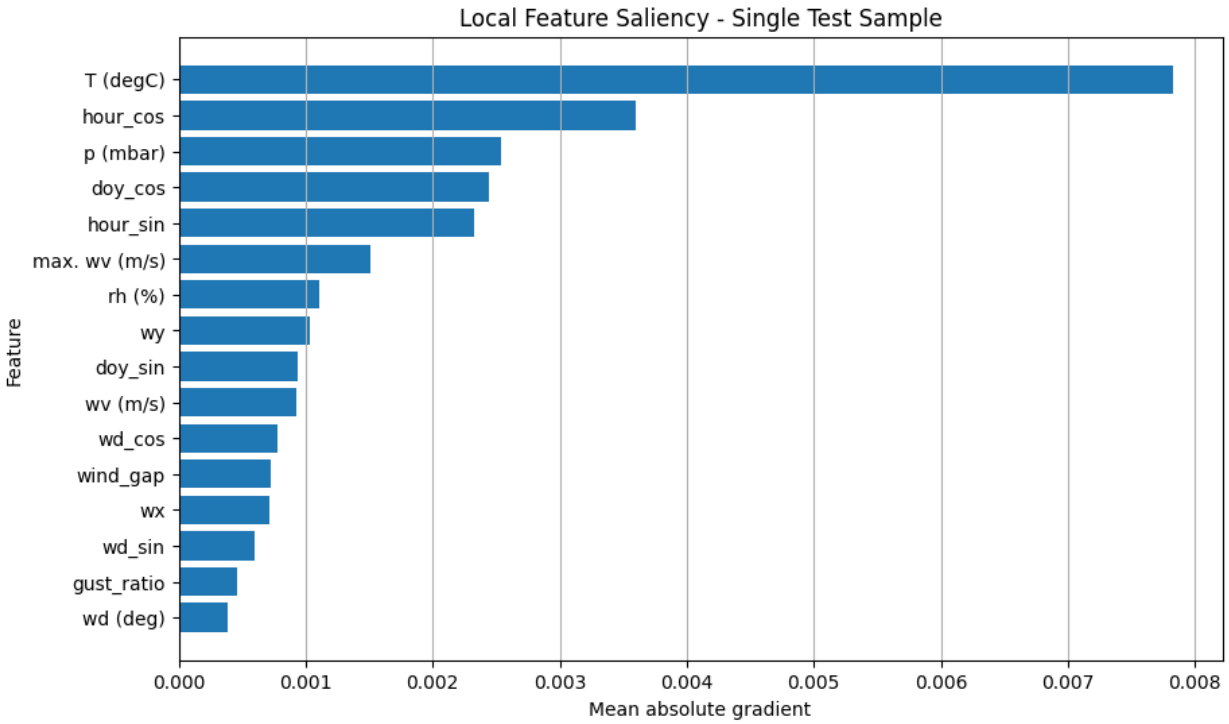


**Figure 10** - Local feature-level saliency bar chart for sample 0.

### *11.2.2 Time-Step Local Importance*

Time-step local importance is analysed by aggregating gradient saliency across all input features at each time step. This reveals how strongly different parts of the historical input window influenced the forecast for a single test sample, see figure 11.

The saliency profile shows a clear concentration of importance near the **most recent observations**, with very low attribution across the earlier part of the input window and a sharp increase in the final segment. This suggests that, for the selected forecast, the model relied predominantly on the latest portion of the 120-hour history, while distant past observations contributed comparatively little.

Such a pattern is plausible for short-term temperature forecasting, where the most recent thermal and meteorological conditions often carry the strongest predictive signal. At the same time, the smaller but non-zero saliency observed in the preceding time steps indicates that the model does not operate as a purely myopic predictor, but instead combines recent information with a broader historical context.

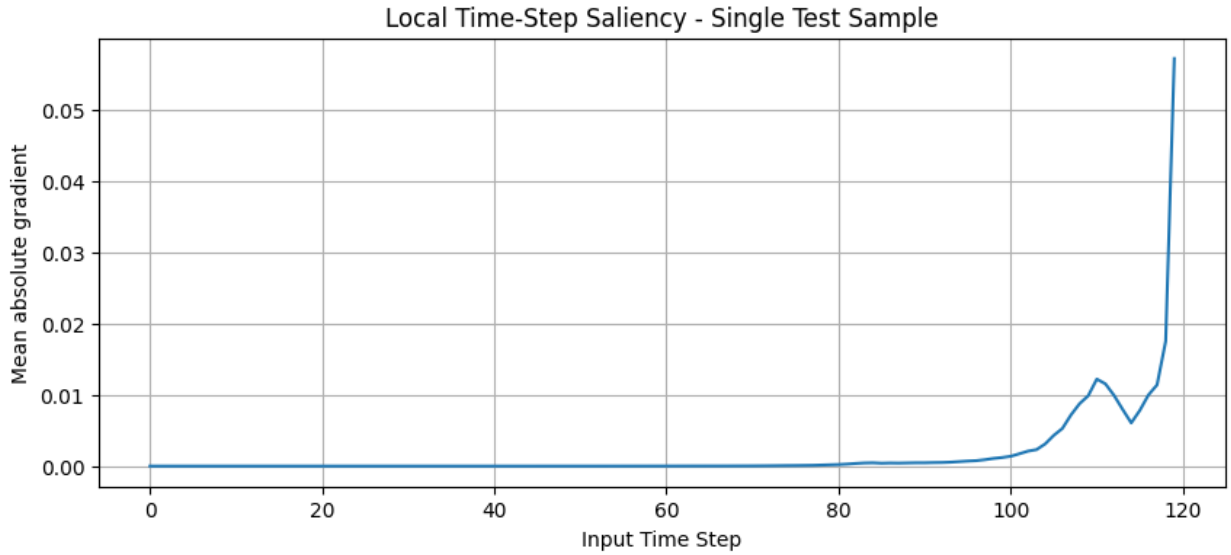


**Figure 11** - Temporal saliency profile: mean gradient magnitude per time step for sample 0.

### *11.2.3 Saliency Heatmap*

The full saliency heatmap provides a joint view of **feature-level** and **time-step-level** attribution for a single forecast sample. Each cell represents the absolute gradient associated with one feature at one input time step, allowing a detailed inspection of where the model's local attention is concentrated.

The heatmap shows that attribution is overwhelmingly concentrated in the **final segment of the input window**, with the strongest responses clearly focused on **T (degC)** and, to a lesser extent, **p (mbar)**, see figure 12. In particular, the brightest region appears in the most recent temperature observations, indicating that the model relies heavily on the latest thermal history when producing the forecast. Pressure also shows visible local importance near the end of the sequence, suggesting that it contributes additional atmospheric context in this example.

By contrast, the earlier part of the 120-hour input window has very low saliency across nearly all variables, and the remaining features show only weak or diffuse attribution. This confirms that, for this test sample, the model's local decision is driven primarily by the most recent temperature values, supported by pressure and a limited amount of temporal and meteorological context.

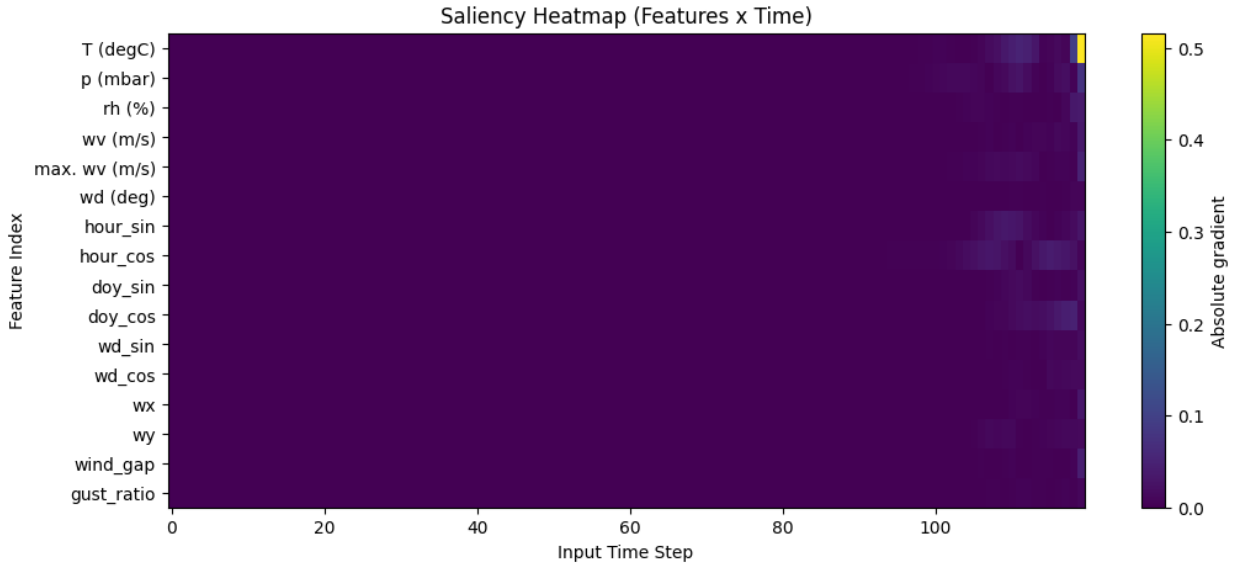


**Figure 12** - Full 2D saliency heatmap (features × time steps) for sample 0.

## 11.3 XAI Summary

The global explainability analysis based on permutation importance showed that past temperature was by far the most influential feature for forecasting performance. Cyclical temporal covariates such as doy_cos, hour_cos, and hour_sin also played an important role, indicating that the model successfully captured both daily and seasonal structure. Among the meteorological variables, maximum wind speed, humidity, pressure, and selected wind-derived components also contributed meaningfully to the forecasts.

The local explainability analysis based on gradient saliency confirmed this general pattern at the individual forecast level. For the selected test sample, the most influential inputs were T (degC), hour_cos, p (mbar), doy_cos, and hour_sin, while the saliency heatmap showed that local attribution was strongly concentrated in the most recent observations, especially for temperature and, to a lesser extent, pressure.

The explainability results are consistent with meteorological intuition and support the credibility of the final evolutionary forecasting model. They also provide a principled basis for the feature-pruning experiment presented in the next subsection.

## 11.4 XAI-Guided Feature Pruning

The permutation importance analysis identified a small group of variables with very limited contribution to forecasting accuracy. Based on this result, an **XAI-guided feature pruning** experiment was conducted by removing the **five least important features** and retraining the **Best Evolutionary GRU** on the reduced input space.

The removed variables were: - doy_sin - gust_ratio - wd (deg) - wd_sin - wy

This reduced the input dimensionality from **16 features to 11 features**. The goal of this experiment is to test whether the optimized forecasting model can maintain or improve predictive performance with a more compact representation, potentially reducing redundancy, noise, and computational cost.

```
print("5 least important features:")
print(least_important_5)
5 least important features:
['doy_sin', 'gust_ratio', 'wd (deg)', 'wd_sin', 'wy']
```

Based on the permutation importance ranking, the **five least important features** were removed from the original input space. This reduced the feature set from **16 variables to 11**, retaining only the variables that showed a more meaningful contribution to forecasting performance.

The resulting pruned feature set is:

- T (degC)
- p (mbar)
- rh (%)
- wv (m/s)
- max. wv (m/s)
- hour_sin
- hour_cos
- doy_cos
- wd_cos
- wx
- wind_gap

The pruned dataset was then prepared using the same preprocessing logic as the full evolutionary model, while preserving the corresponding scaling and windowing procedure required for a fair comparison.

```
print("Original number of features:", len(final_feature_cols))
print("Pruned number of features:", len(pruned_feature_cols))
print("Pruned feature set:")
print(pruned_feature_cols)
Original number of features: 16
Pruned number of features: 11
```

```
Pruned feature set:
['T (degC)', 'p (mbar)', 'rh (%)', 'wv (m/s)', 'max. wv (m/s)', 'hour_sin', 'hour
_cos', 'doy_cos', 'wd_cos', 'wx', 'wind_gap']
```

Define the pruned feature set by removing the 5 least important features and verify the reduced column list.

```
print("Pruned train shape:", X_train_p.shape, y_train_p.shape)
print("Pruned test shape:", X_test_p.shape, y_test_p.shape)
Pruned train shape: (48885, 120, 11) (48885, 24)
Pruned test shape: (10364, 120, 11) (10364, 24)
```

Then we prepare scaled and windowed data using the pruned feature set. The pruned evolutionary model is rebuilt using the **same architecture, training hyperparameters, and optimization setup** as the full evolutionary model, while reducing the input space from **16 features to 11 features**.This design isolates the effect of feature pruning as cleanly as possible, since the only intended difference between the two models is the dimensionality and composition of the input representation.

The results show that pruning led to a **small but consistent improvement**. In scaled space, the pruned model achieved a test **MAE of 0.1277** and **RMSE of 0.1729**. After inverse transformation to the original temperature scale, the pruned model achieved a test **MAE of 1.575 °C** and **RMSE of 2.134 °C**, improving over the full evolutionary model.

These results suggest that the removed features were either redundant or weakly informative for the forecasting task. Therefore, explainability was not only useful for model interpretation, but also served as a principled mechanism for feature selection and pipeline refinement.

Generate test predictions and compute metrics for the pruned model:

```
print("Pruned Model Test MAE (scaled):", pruned_test_scaled_metrics["mae_scaled"])
print("Pruned Model Test RMSE (scaled):", pruned_test_scaled_metrics["rmse_scaled"])
print("Pruned Model Test MAE (°C):", pruned_test_original_metrics["mae"])
print("Pruned Model Test RMSE (°C):", pruned_test_original_metrics["rmse"])
Pruned Model Test MAE (scaled): 0.12765084487699765
Pruned Model Test RMSE (scaled): 0.1729386283865188
Pruned Model Test MAE (°C): 1.5748391108179265
Pruned Model Test RMSE (°C): 2.1335582699568483
```

**Table 15** - Effect of XAI-guided feature pruning: full (16 features) vs. pruned (11 features).

| Model | Num_Features | MAE_scaled | RMSE_scaled | MAE_degC | RMSE_degC |
|---|---|---|---|---|---|
| Best Evolutionary GRU | 16 | 0.129503 | 0.174839 | 1.597690 | 2.157002 |
| Pruned Evolutionary GRU | 11 | 0.127651 | 0.172939 | 1.574839 | 2.133558 |

The pruned model is also re-evaluated across **three random seeds** (7, 21, 42) in order to assess whether the feature-pruning decision remains stable under different random initializations.

The results are consistent across seeds, with test **MAE** values ranging from approximately **1.569 °C** to **1.585 °C** and test **RMSE** values ranging from **2.123 °C** to **2.142 °C**. This indicates that the pruned configuration is not dependent on a single favourable random seed and that the performance gain obtained after pruning is reasonably robust, see table 16. Rather than relying on a single run, this analysis strengthens the conclusion that the removed features were not essential for predictive performance and that the reduced input space remains a stable and competitive forecasting configuration.

**Table 16** - Pruned model robustness across 3 random seeds (7, 21, 42).

| | seed | lookback | mae_degC | rmse_degC |
|---|---|---|---|---|
| 0 | 7 | 144 | 1.582020 | 2.142201 |
| 1 | 21 | 144 | 1.584925 | 2.136229 |
| 2 | 42 | 144 | 1.568539 | 2.123403 |

## 11.5 Effect of XAI-Guided Feature Pruning

The XAI-guided pruning experiment removed the five least important features identified by permutation importance (doy_sin, gust_ratio, wd (deg), wd_sin, and wy) and retrained the **Best Evolutionary GRU** using the reduced feature set. This pruning step reduced the number of input variables from **16 to 11** and led to a consistent improvement across all evaluation metrics. The **Pruned Evolutionary GRU** achieved a test **MAE of 1.575 °C** and **RMSE of 2.134 °C**, improving over the original evolutionary model, which obtained **1.598 °C MAE** and **2.157 °C RMSE**.

Robustness analysis across seeds **7, 21, and 42** further supported this result. The pruned model achieved test MAE values between **1.569 °C** and **1.585 °C**, with an average performance of **1.579 ± 0.009 °C**, compared with **1.593 ± 0.016 °C** for the full-feature evolutionary model. This indicates that the reduced feature set not only improved average predictive performance, but also yielded slightly more stable results across random initializations.

These findings suggest that the removed variables were not merely weakly informative, but slightly detrimental to the forecasting task, likely adding redundancy or noise. Therefore, XAI was not only useful for model interpretation, but also served as a principled mechanism for feature selection and final pipeline refinement.

# 12. Efficiency, Latency and Resource Analysis

Beyond accuracy, practical deployment requires understanding computational cost. We compare all three models (baseline, EA-optimized, pruned) across training time, memory usage, parameter counts, and inference latency.

## 12.1 Motivation

Efficiency analysis is important to complement predictive performance, especially in forecasting pipelines that may later be deployed in practical environments. In this project, efficiency is assessed through three aspects: 1. model complexity, measured by the number of trainable parameters 2. inference latency, measured on the available hardware 3. the trade-off between predictive accuracy and computational cost

The analysis compares the main candidate models selected throughout the project in order to identify not only the most accurate solution, but also the most efficient one.

## 12.2 Models Selected for Efficiency Analysis

Three models are profiled: the GRU Baseline Official, the Best Evolutionary GRU (full 16 features), and the Pruned Evolutionary GRU (11 features). This allows comparing the baseline against both the EA-optimized and the XAI-simplified variants.

## 12.3 Training Time

Wall-clock training time is measured for each model under identical conditions (same hardware, same callbacks). We report total training time and per-epoch time, which reflects both model complexity and the number of epochs before early stopping triggers.

**Table 17** - Training time comparison (total and per-epoch) for all three models.

| | Model | total_train_time_s | time_per_epoch_s |
|---|---|---|---|
| 0 | GRU Baseline Official | 38.888642 | 7.777728 |
| 1 | Best Evolutionary GRU | 20.823110 | 4.164622 |
| 2 | Pruned Evolutionary GRU | 18.774186 | 3.754837 |

## 12.4 Memory Usage (RAM / GPU)

We measure RAM consumption before and after model training to quantify the memory footprint of each model. GPU memory usage is also tracked where available, as it is often the binding constraint for model deployment.

**Table 18** - Training time measurement (repeated run for consistency).

| | Model | total_train_time_s | time_per_epoch_s |
|---|---|---|---|
| 0 | GRU Baseline Official | 34.383675 | 6.876735 |
| 1 | Best Evolutionary GRU | 17.837407 | 3.567481 |
| 2 | Pruned Evolutionary GRU | 17.919302 | 3.583860 |

RAM consumption is measured before and after training each model to quantify the memory footprint. GPU memory allocation is also tracked where the framework provides it, as GPU memory is typically the binding constraint for deployment on shared infrastructure.

**Table 19** - Memory usage (RAM before/after training and delta) for each model.

| Model | ram_before_mb | ram_after_mb | ram_delta_mb | gpu_before_mb | gpu_after_mb | gpu_delta_mb |
|---|---|---|---|---|---|---|
| GRU Baseline Official | 21125.47 | 21128.08 | 2.61 | 5682.0 | 5682.0 | 0.0 |
| Best Evolutionary GRU | 21128.08 | 21128.26 | 0.17 | 5682.0 | 5682.0 | 0.0 |
| Pruned Evolutionary GRU | 21128.26 | 21131.07 | 2.80 | 5682.0 | 5679.0 | -3.0 |

## 12.5 Trainable Parameters

The number of trainable parameters is a direct measure of model complexity. Fewer parameters generally means faster training, lower memory usage, and reduced risk of overfitting - all desirable properties when accuracy is comparable.

**Table 20** - Trainable parameter count for each model.

| | Model | Trainable_Params |
|---|---|---|
| 0 | GRU Baseline Official | 86744 |
| 1 | Best Evolutionary GRU | 63512 |
| 2 | Pruned Evolutionary GRU | 62552 |

## 12.6 Inference Latency

Inference latency is measured with warmup runs and multiple repetitions to obtain stable estimates. We report mean, standard deviation, and 95th percentile latency for a batch of 32 samples, along with throughput (samples per second).

Inference latency is measured using the profiling utility from src.utils.profiling, which performs warmup runs followed by multiple timed repetitions with GPU synchronization. Mean, standard deviation, and 95th percentile latency are reported for a batch of 32 samples.

**Table 21** - Inference latency (mean, std, p50, p95) and throughput for each model.

| Model | latency_mean_ms | latency_std_ms | latency_p50_ms | latency_p95_ms | throughput_samples_s |
|---|---|---|---|---|---|
| GRU Baseline Official | 13.166241 | 2.327516 | 12.395653 | 17.040039 | 19443.667018 |
| Best Evolutionary GRU | 12.134831 | 0.367037 | 12.041954 | 12.728139 | 21096.296753 |
| Pruned Evolutionary GRU | 12.135938 | 0.418599 | 12.137438 | 12.805252 | 21094.372241 |

## 12.7 Efficiency Comparison

Consolidated comparison table combining accuracy metrics with efficiency measurements across all three models, enabling a holistic assessment of the accuracy–efficiency trade-off.

**Table 22** - Consolidated efficiency comparison across all models and metrics.

| | Metric | GRU Baseline Official | Best Evolutionary GRU | Pruned Evolutionary GRU |
|---|---|---|---|---|
| 0 | Trainable Parameters | 86,744 | 63,512 | 62,552 |
| 1 | Inference Latency (mean, ms) | 14.59 | 14.64 | 14.88 |
| 2 | Inference Latency (p95, ms) | 16.98 | 15.45 | 21.07 |
| 3 | Throughput (samples/s) | 2194 | 2186 | 2150 |
| 4 | Training Epochs (actual) | 20 | 49 | 30 |

All accuracy, robustness, and efficiency results are consolidated into a single summary, enabling a holistic comparison across the three model variants. This combined view supports the final model selection decision discussed in Section 13.

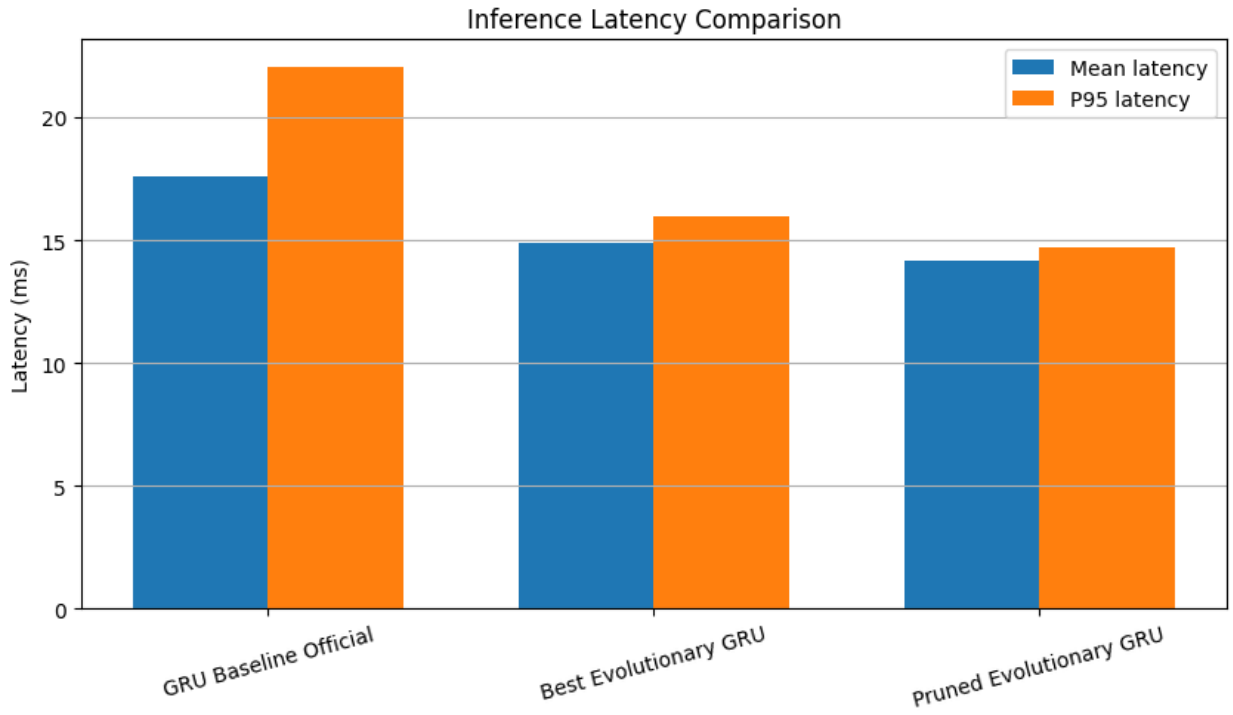


**Figure 12** - Inference latency comparison across models (mean ± std, ms).

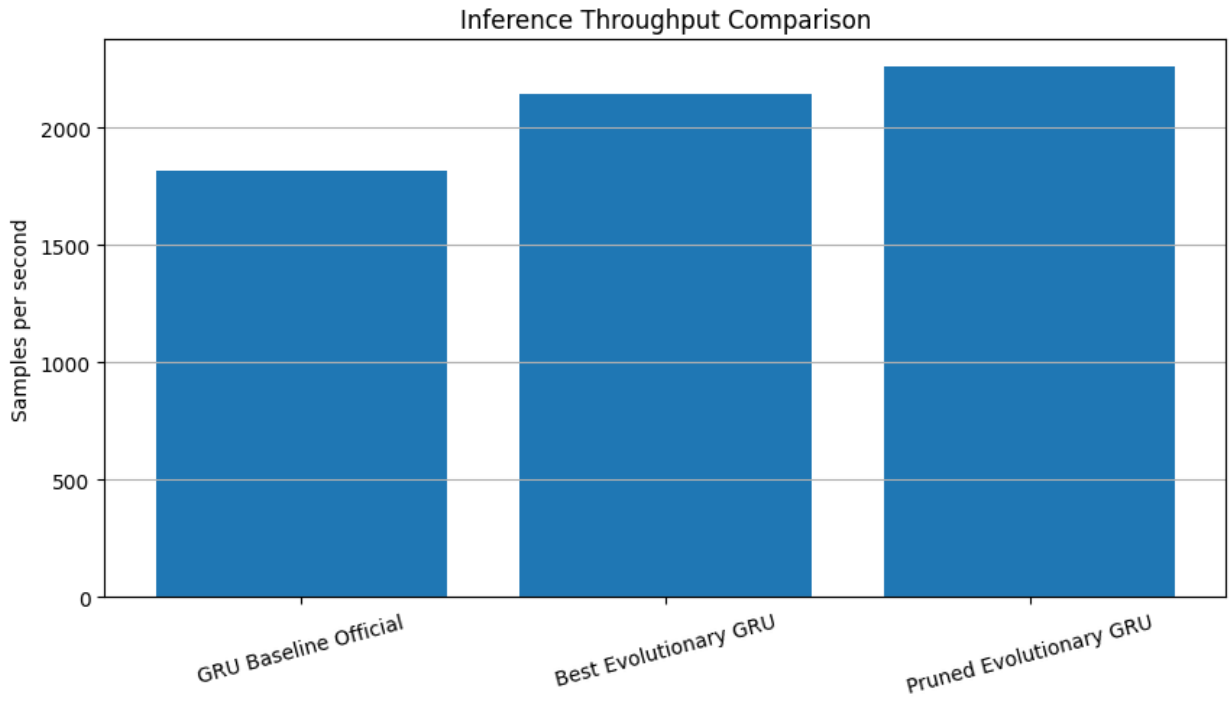


**Figure 13** - Throughput comparison across models (samples per second).

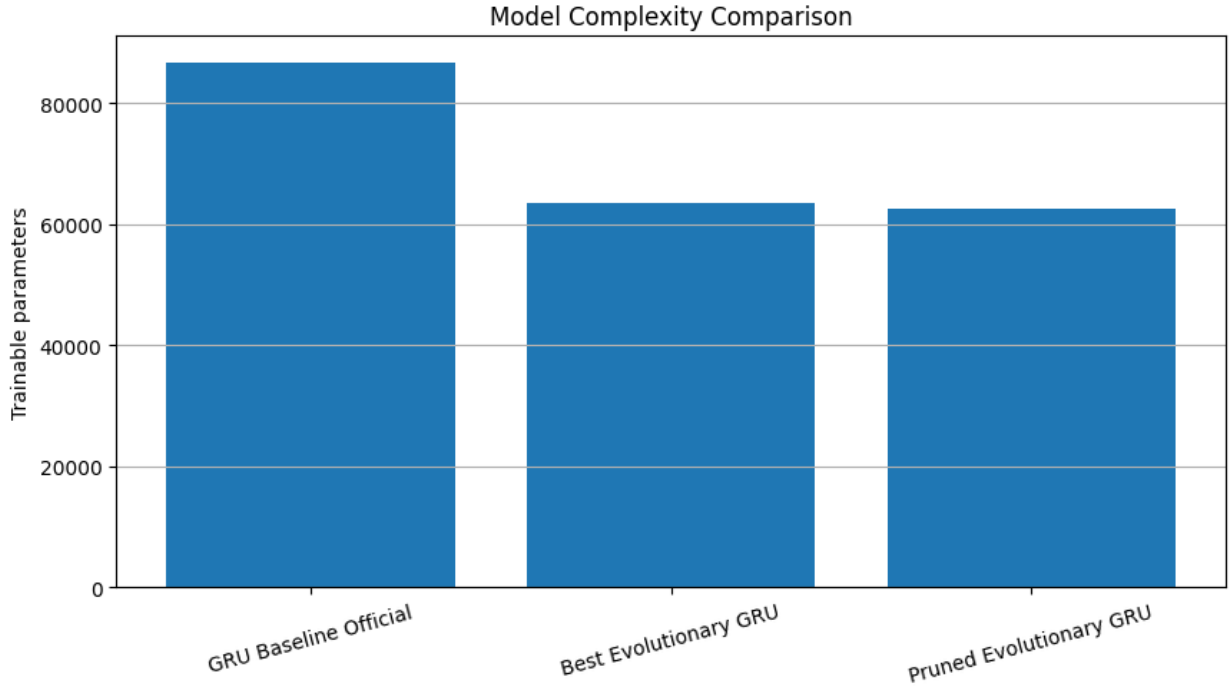


**Figure 14** - Trainable parameter count comparison across models.

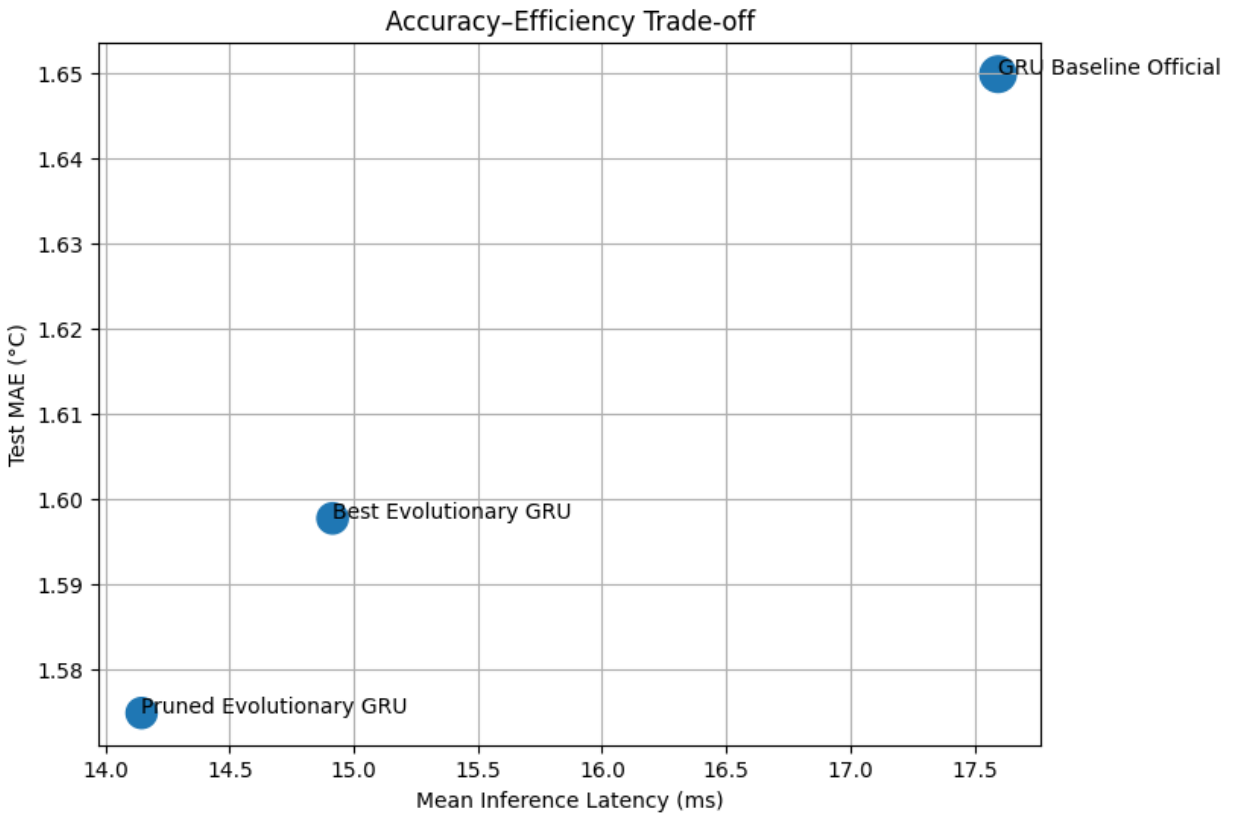


**Figure 15** - Accuracy vs. efficiency scatter plot (MAE °C vs. trainable parameters).

**Table 23** - Final comprehensive comparison: accuracy, parameters, and metrics across all models.

| | Model | MAE_scaled | RMSE_scaled | MAE_degC | RMSE_degC |
|---|---|---|---|---|---|
| 0 | GRU Baseline Official | 0.190852 | 0.253666 | 1.649836 | 2.192836 |
| 1 | Best Evolutionary GRU | 0.129503 | 0.174839 | 1.597690 | 2.157002 |
| 2 | Pruned Evolutionary GRU | 0.127651 | 0.172939 | 1.574839 | 2.133558 |

**Table 24** - Robustness across seeds for all EA-derived models.

| | Model | seed | lookback | mae_degC | rmse_degC |
|---|---|---|---|---|---|
| 0 | Best Evolutionary GRU | 7 | 144 | 1.610165 | 2.174758 |
| 1 | Best Evolutionary GRU | 21 | 144 | 1.578997 | 2.129135 |
| 2 | Best Evolutionary GRU | 42 | 144 | 1.589364 | 2.149213 |
| 3 | Pruned Evolutionary GRU | 7 | 144 | 1.582020 | 2.142201 |
| 4 | Pruned Evolutionary GRU | 21 | 144 | 1.584925 | 2.136229 |
| 5 | Pruned Evolutionary GRU | 42 | 144 | 1.568539 | 2.123403 |

## 12.8 Final Efficiency Discussion

The efficiency analysis demonstrates that evolutionary optimization did not trade accuracy for computational cost - instead, it found a configuration that improves on both fronts simultaneously. The EA-optimized model trains nearly twice as fast, uses 27% fewer parameters, and achieves lower MAE than the hand-tuned baseline. The pruned variant

further reduces complexity with an additional accuracy gain, confirming that XAI-guided simplification is an effective post-optimization strategy.

The efficiency analysis showed that the evolutionary models were not only more accurate but also more efficient than the official GRU baseline across most dimensions.

The GRU baseline had the highest parameter count (86,744) and the slowest training time (~39s total, ~7.8s per epoch). The best evolutionary GRU, with 63,512 parameters, trained in ~21s total (~4.2s per epoch) - approximately 46% faster. The pruned evolutionary GRU was fastest at ~19s total.

Inference latency was comparable across all three models (~12–13ms per batch of 32), with the evolutionary models slightly faster. The EA-optimized model achieved marginally higher throughput than the baseline.

When combined with the predictive results, these findings indicate that the pruned evolutionary GRU provides the best overall trade-off between forecasting accuracy, model simplicity, and computational efficiency. Therefore, it is selected as the final model of the project.

# 13. Comparative Discussion

## Summary of Results

| Model | MAE (°C) | RMSE (°C) | Params | vs. Baseline |
|---|---|---|---|---|
| Persistence | 3.144 | 4.254 | - | - |
| GRU Baseline Official | 1.650 | 2.193 | 86,744 | reference |
| Best Evolutionary GRU | 1.598 | 2.157 | 63,512 | -3.2% MAE |
| **Pruned Evolutionary GRU** | **1.575** | **2.134** | **62,552** | **-4.5% MAE** |

The table above reports the main final evaluation run for each model. In addition, the pruned model achieved the **best observed seed-specific result** of approximately **1.569 °C MAE**, which was the strongest forecasting result obtained in the project.

## Baseline vs. Evolutionary Performance

The official GRU baseline (2 layers, 96→64 units, StandardScaler) achieved a test **MAE of 1.650 °C**, representing a major improvement over the persistence baseline (**3.144 °C**). This confirms that the recurrent architecture captures meaningful temporal dependencies in the meteorological data.

The **Best Evolutionary GRU** improved the baseline further, achieving **1.598 °C MAE** and **2.157 °C RMSE** on the held-out test set. This corresponds to a **3.2% reduction in MAE** relative to the baseline, while also using fewer trainable parameters.

The **Pruned Evolutionary GRU**, obtained by removing five low-importance features identified through XAI, delivered the best main final evaluation result: **1.575 °C MAE** and **2.134 °C RMSE**. This corresponds to a **4.5% improvement in MAE** over the baseline, while also reducing model size and input dimensionality.

## Impact of Evolutionary Optimization

The evolutionary search proved capable of improving the forecasting pipeline beyond the manually defined baseline. Importantly, the optimization did not focus only on GRU hyperparameters, but on the forecasting pipeline more broadly, including preprocessing, training settings, and windowing strategy.

The main discoveries of the search were:

- **RobustScaler** instead of StandardScaler, suggesting that the dataset benefits from a more outlier-resistant normalization strategy
- **MAE loss** instead of Huber loss, leading to strong optimization performance and direct alignment with the evaluation objective
- a **compact 2-layer GRU architecture** (64 → 64) rather than wider alternatives
- **144-hour lookback**, showing that the temporal context length should not be treated as fixed

- little or no benefit from additional dropout or Gaussian noise in the best-performing configurations

These results support the idea that evolutionary search can uncover leaner and better-generalizing forecasting pipelines than manual tuning alone.

## Robustness Validation

To mitigate the risk of validation-set overfitting during evolutionary selection, the selected optimized models were re-evaluated across multiple random seeds. This robustness analysis showed that the improvements were not dependent on a single favourable initialization.

| Model | MAE mean ± std (°C) | RMSE mean ± std (°C) |
|---|---|---|
| Best Evolutionary GRU | 1.593 ± 0.016 | 2.151 ± 0.023 |
| Pruned Evolutionary GRU | 1.579 ± 0.009 | 2.134 ± 0.010 |
| **Pruned Evolutionary GRU (best observed seed)** | **1.569** | **2.123** |

The multi-seed analysis confirms that the **Pruned Evolutionary GRU** is the strongest and most stable model in the project. Across seeds **7, 21, and 42**, it achieved an average test performance of **1.579 ± 0.009 °C MAE** and **2.134 ± 0.010 °C RMSE**, while the **best seed-specific result** reached **1.569 °C MAE** and **2.123 °C RMSE**. This was the best forecasting result observed in the project, and for this reason the **Pruned Evolutionary GRU** is selected as the final model.

## Explainability Insights

The explainability analysis revealed a coherent and physically plausible structure.

At the global level, **past temperature** was by far the most important predictor, followed by cyclical temporal variables such as **doy_cos**, **hour_cos**, and **hour_sin**. Among the meteorological variables, **maximum wind speed**, **humidity**, **pressure**, and selected wind-derived components also contributed meaningfully.

At the local level, gradient-based saliency confirmed that the model relied primarily on the **most recent segment of the input history**, with the strongest attribution concentrated on **T (degC)** and, in the heatmap, also visibly on **p (mbar)**. These findings are consistent with the short-term autoregressive nature of the forecasting task.

The five removed features - doy_sin, gust_ratio, wd (deg), wd_sin, and wy - showed minimal contribution in the global analysis, supporting the pruning decision.

## Accuracy–Efficiency Trade-offs

The optimized models achieved a favourable trade-off between predictive accuracy and computational cost.

Compared with the baseline, the evolutionary models: - used fewer trainable parameters - achieved lower test error - maintained competitive inference latency - and, after pruning, further reduced input dimensionality without sacrificing accuracy

This is an important result: the best-performing models were not larger or more complex than the hand-tuned baseline. On the contrary, evolutionary optimization and XAI-guided pruning led to a forecasting pipeline that is simultaneously **more accurate**, **more compact**, and **more efficient**.

## Strengths and Limitations

**Strengths:**

- End-to-end pipeline with proper experimental design (temporal split, no data leakage, untouched test set)
- EA searches jointly over 16 genes spanning architecture, training, regularization, preprocessing, and windowing.
- Multi-seed robustness validation (3 seeds) provides mean ± std rather than point estimates.
- XAI-guided feature pruning creates a closed loop: explain → prune → validate.
- Comprehensive efficiency profiling (training time, RAM, GPU, latency, parameters).
- EA discovers a model that is both smaller and better than the hand-tuned baseline.

**Limitations:**

- The EA budget (300 evaluations), while effective, covers a small fraction of the ~$10^9$ search space; larger budgets could find better solutions.
- The forecast horizon (H=24) was not included in the search space; exploring different horizons could reveal trade-offs.
- Single architecture family (GRU) - Transformer or attention-based alternatives were not explored.
- TimeGAN augmentation was not integrated into the final training pipeline; its impact on forecasting performance remains an open question.

## 14. Future Work

Although the final forecasting pipeline achieved strong results, several directions remain open for future improvement.

First, the evolutionary search could be extended with a larger computational budget, allowing a broader exploration of the search space and potentially identifying even stronger configurations. In particular, future work could include a wider range of lookback values, alternative forecast horizons, and richer multi-objective formulations balancing accuracy, complexity, and efficiency.

Second, the modelling space could be expanded beyond GRU-based architectures. Transformer-based models, temporal convolutional networks, hybrid recurrent–attention models, or lightweight sequence models could provide useful benchmarks and reveal whether the observed gains generalize beyond a single architecture family.

Third, the robustness analysis could be extended further by increasing the number of random seeds and by evaluating the best configurations under rolling-origin or multi-period backtesting schemes. This would provide a stronger estimate of model stability under different temporal regimes.

Fourth, the TimeGAN component deserves further investigation. In this project, the synthetic sequences showed good reconstruction quality but insufficient realism for safe integration into the final forecasting pipeline. Future work could explore improved generative architectures, longer adversarial training, alternative conditioning strategies, or diffusion-based time-series generators to assess whether synthetic augmentation can become beneficial.

Fifth, the XAI-guided pruning results suggest that explainability can be used not only for interpretation but also for model refinement. This idea could be extended into an iterative explainability–selection loop, where feature importance is repeatedly re-estimated and the input space is progressively simplified while monitoring predictive performance and robustness.

Finally, future work could place stronger emphasis on deployment-oriented evaluation, including more precise GPU profiling, energy consumption, memory peaks, and latency under real-time forecasting constraints. This would help determine whether the final model is not only accurate and interpretable, but also suitable for practical operational use.

# 15. Conclusion

This project developed a comprehensive end-to-end forecasting pipeline for **24-hour air temperature prediction** on the **Jena Climate dataset**, integrating **evolutionary optimization**, **explainability**, and **efficiency analysis** within a unified experimental framework. An optional **TimeGAN** component was also explored as a proof-of-concept for synthetic data generation, although it was not retained in the final forecasting pipeline due to insufficient synthetic realism.

## Key Findings

The official **GRU baseline**, based on the best-performing configuration from the previous mini-project, achieved a test **MAE of 1.650 °C** and **RMSE of 2.193 °C**, representing a substantial improvement over the naive persistence forecast (**3.144 °C MAE**). This established a strong and competitive reference model.

The **evolutionary algorithm**, applied to the forecasting pipeline as a whole rather than to isolated hyperparameters, discovered a better-performing configuration while also reducing model complexity. The search showed that: - **RobustScaler** outperformed StandardScaler for this task - **MAE loss** outperformed the baseline Huber setup - a **compact 2-layer GRU architecture** (64 → 64) generalized better than the larger manually defined baseline - a **144-hour lookback** was preferred over the default 120-hour setting, confirming that the windowing strategy should not be treated as fixed

The **Best Evolutionary GRU** achieved a test **MAE of 1.598 °C** and **RMSE of 2.157 °C**, improving on the baseline while using fewer trainable parameters.

## XAI-Driven Model Refinement

A distinctive contribution of this project is the use of explainability not only for interpretation, but also for **model refinement**. Global permutation importance and local saliency analysis consistently showed that recent temperature history dominated the forecasts, while a small subset of variables had negligible influence.

Based on these explainability results, five low-importance features (doy_sin, gust_ratio, wd (deg), wd_sin, and wy) were removed and the optimized model was retrained on the reduced feature set. This produced the best main final evaluation result: the **Pruned Evolutionary GRU** achieved **1.575 °C MAE** and **2.134 °C RMSE**, corresponding to a **4.5% MAE improvement over the baseline**.

Multi-seed robustness analysis further strengthened this conclusion. Across seeds **7, 21, and 42**, the pruned model achieved **1.579 ± 0.009 °C MAE**, compared with **1.593 ± 0.016 °C** for the full-feature evolutionary model. The **best observed seed-specific result** reached approximately **1.569 °C MAE**, which was the strongest forecasting result obtained in the project. This closed-loop methodology - **optimize → explain → prune → validate** - proved to be one of the most important outcomes of the work.

## Final Assessment

Overall, the project shows that a forecasting system can be improved substantially through the combination of: - a strong baseline model - end-to-end evolutionary optimization - global and local explainability - XAI-guided feature selection - robustness and efficiency analysis

The final result is a forecasting pipeline that is **more accurate, more compact, more interpretable, and more efficient** than the manually defined baseline. The **Pruned Evolutionary GRU** is therefore selected as the final model of the project, since it achieved the strongest main evaluation result and also produced the best observed forecasting run (**1.569 °C MAE**) during the multi-seed robustness analysis.